Compiler Construction

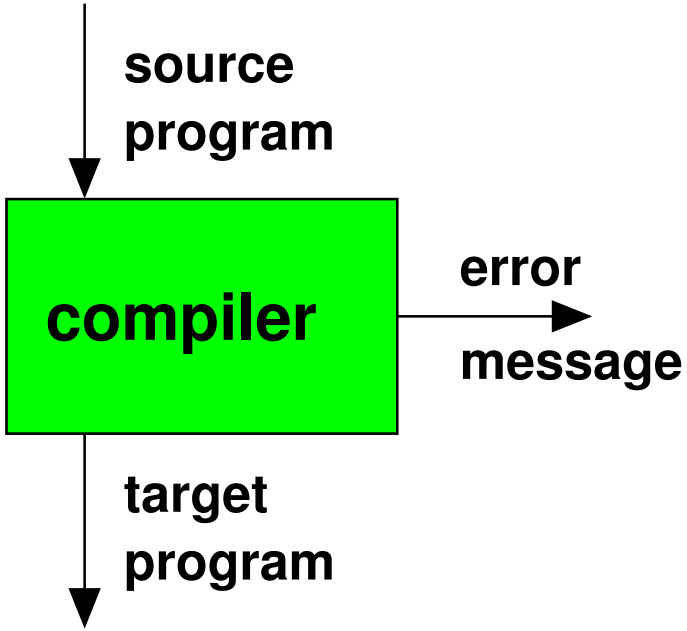
Chapter 2

Compilers at a Glance

Clemens Grelck

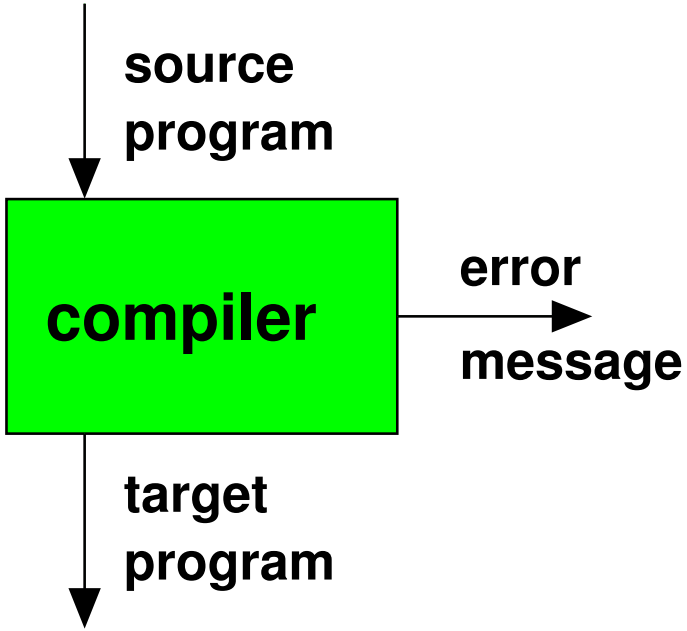
Friedrich Schiller University Jena

Compiler: The User's View



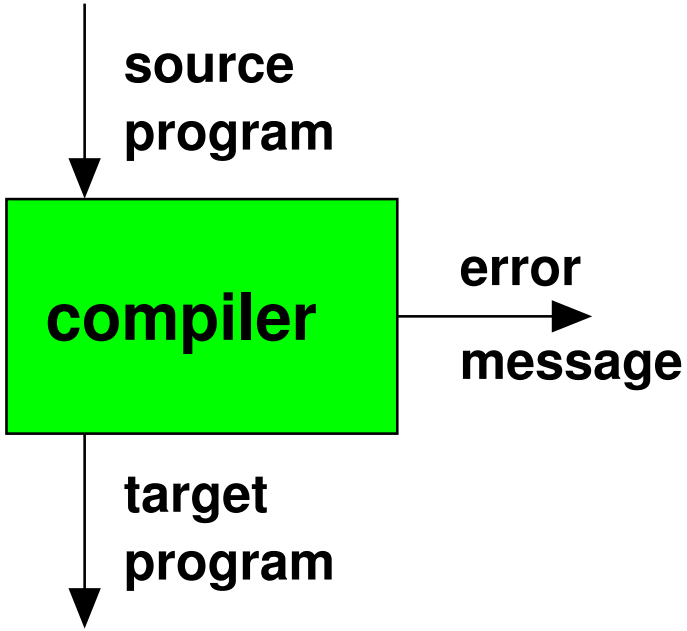
- Compiler reads source program.

Compiler: The User's View



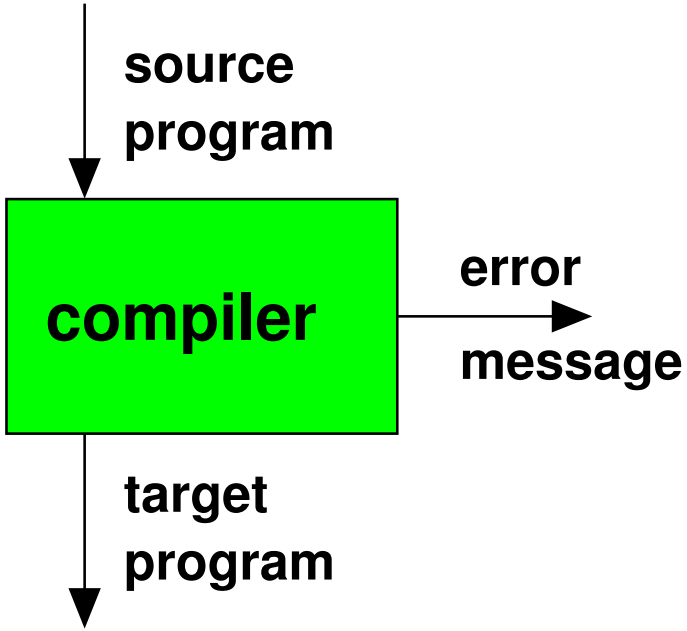
- Compiler reads source program.
- Compiler runs an unreasonable amount of time.

Compiler: The User's View



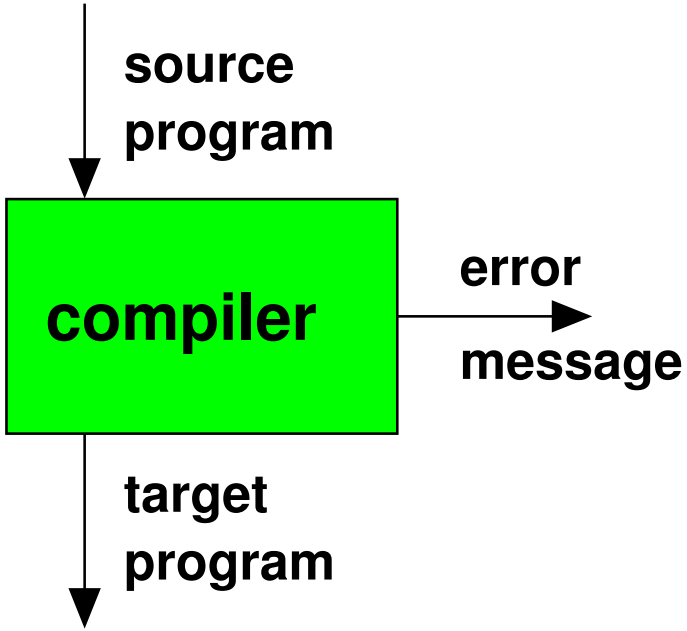
- Compiler reads source program.
- Compiler runs an unreasonable amount of time.
- Compiler terminates with error message.

Compiler: The User's View



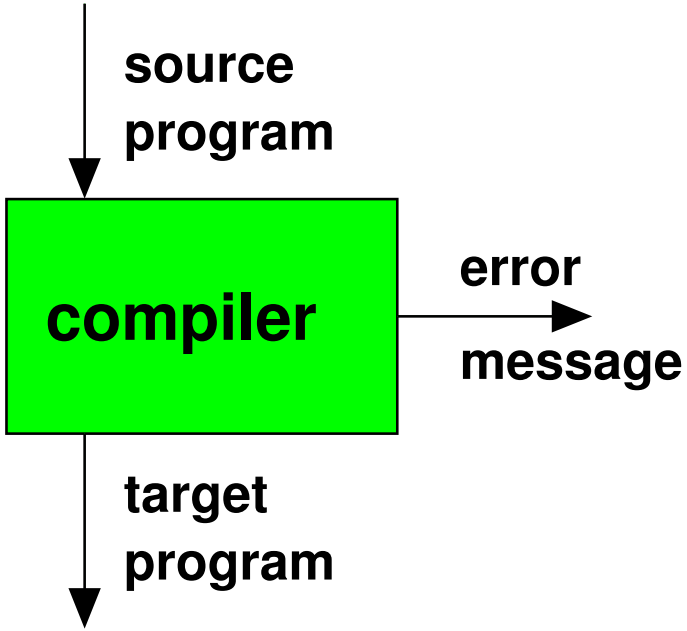
- Compiler reads source program.
- Compiler runs an unreasonable amount of time.
- Compiler terminates with error message.
- User scratches head, edits source program, and ....

Compiler: The User's View



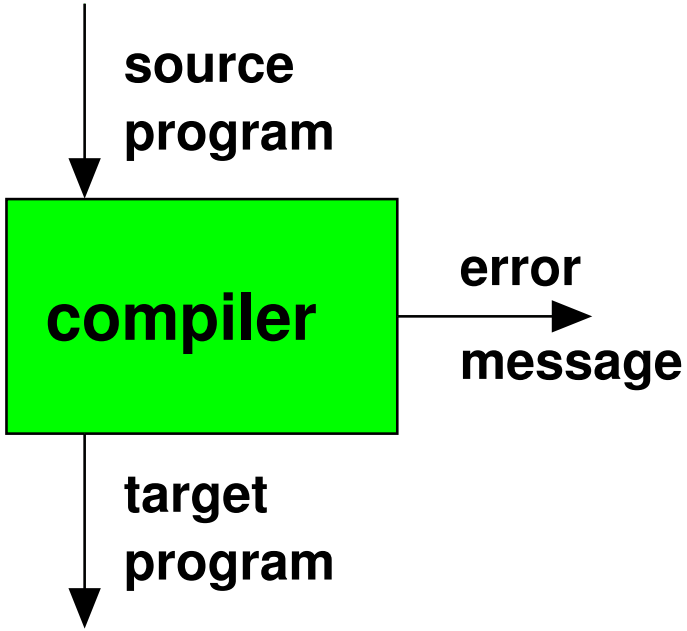
- Compiler reads source program.
- Compiler runs an unreasonable amount of time.
- Compiler terminates with error message.
- User scratches head, edits source program, and ....
- Starts all over again.

Compiler: The User's View



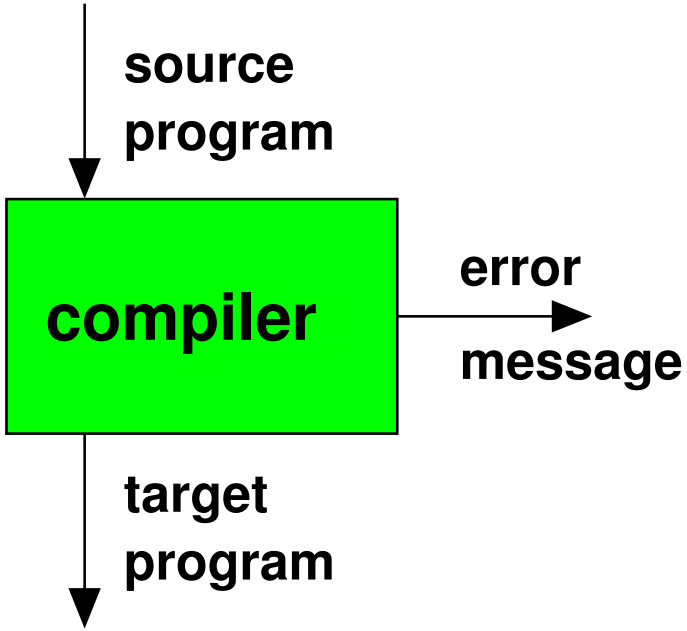
- Compiler reads source program.

Compiler: The User's View



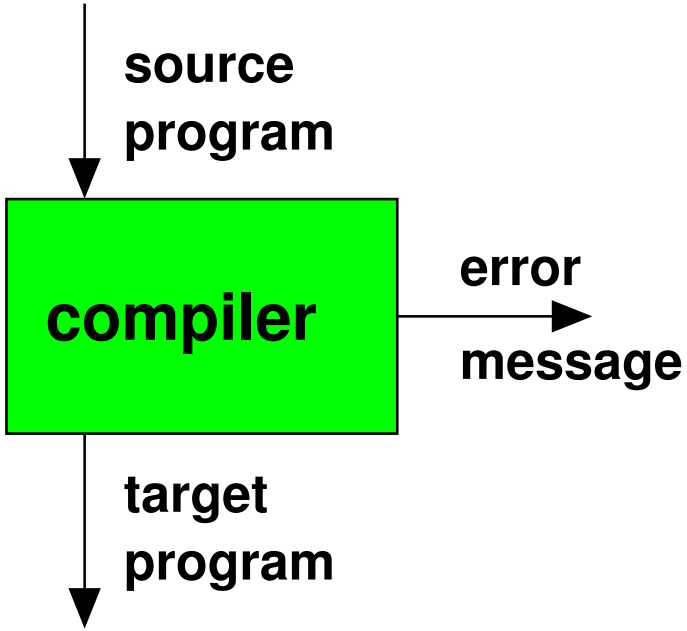
- Compiler reads source program.
- Compiler runs an unreasonable amount of time.

Compiler: The User's View



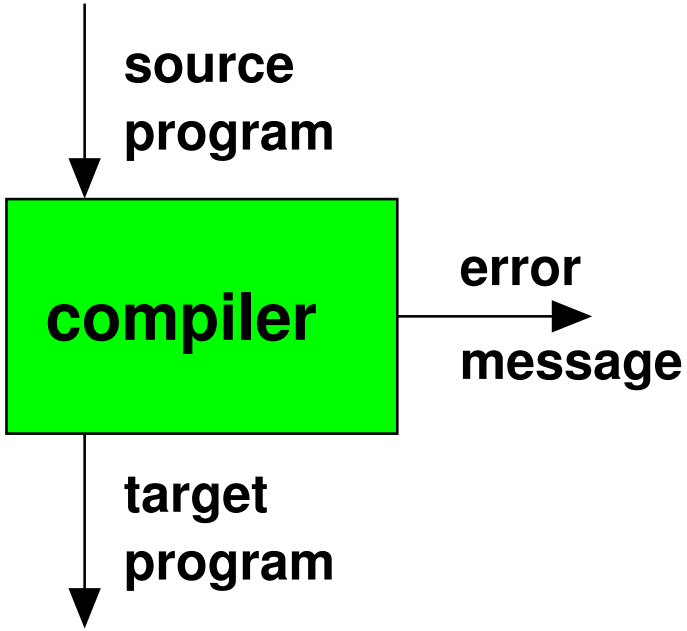
- Compiler reads source program.
- Compiler runs an unreasonable amount of time.
- Compiler terminates with error message **again**.

Compiler: The User's View



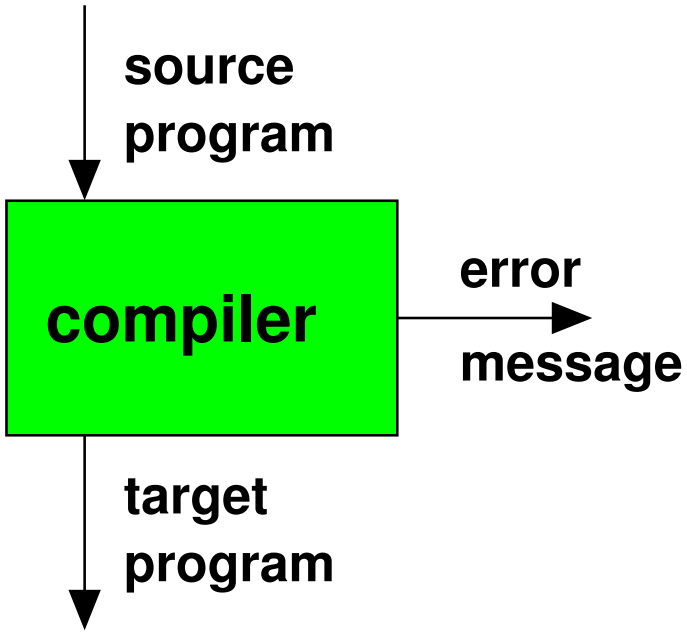
- Compiler reads source program.
- Compiler runs an unreasonable amount of time.
- Compiler terminates with error message **again**.
- User scratches head, edits source program, and ....

Compiler: The User's View



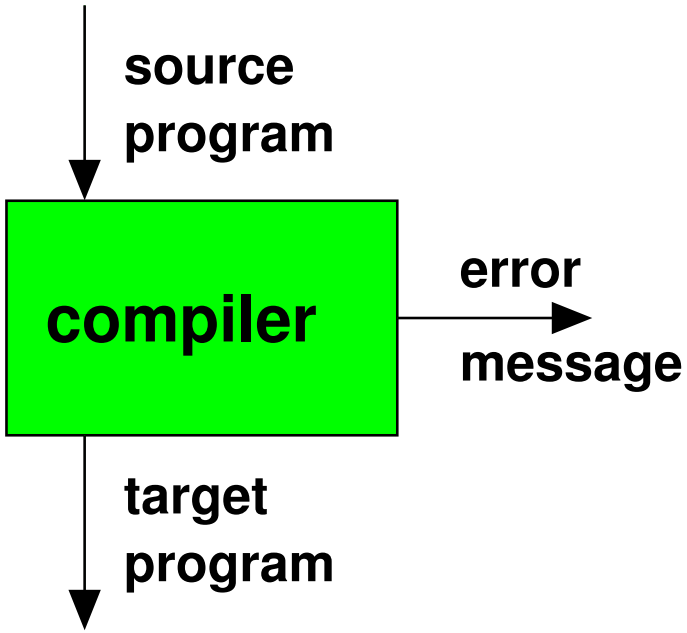
- Compiler reads source program.
- Compiler runs an unreasonable amount of time.
- Compiler terminates with error message **again**.
- User scratches head, edits source program, and ....
- Starts all over again.

Compiler: The User's View



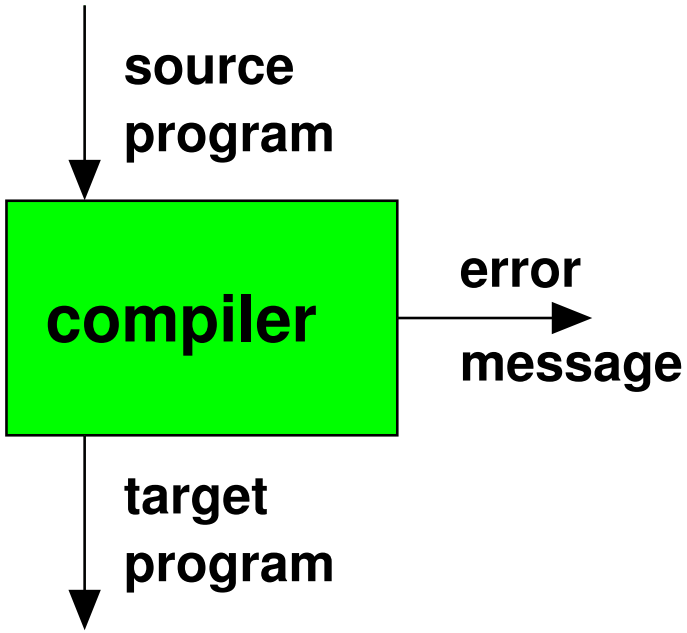
- Compiler reads source program.

Compiler: The User's View



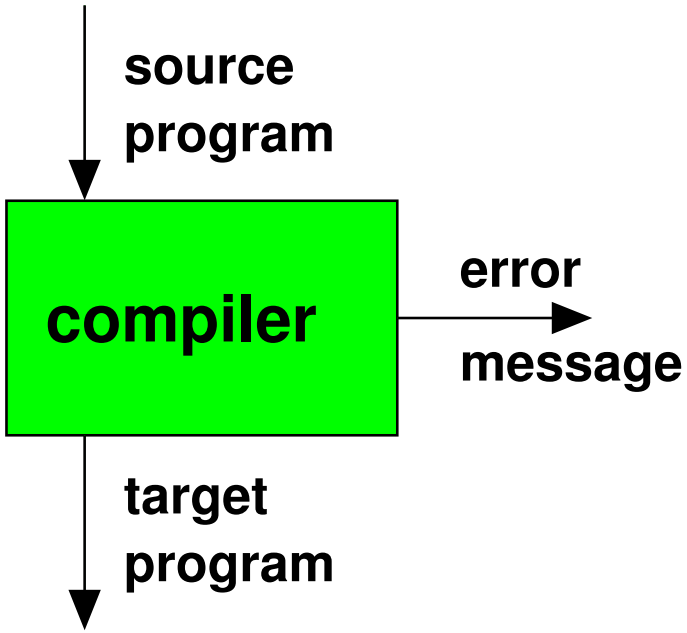
- Compiler reads source program.
- Compiler runs an unreasonable amount of time.

Compiler: The User's View



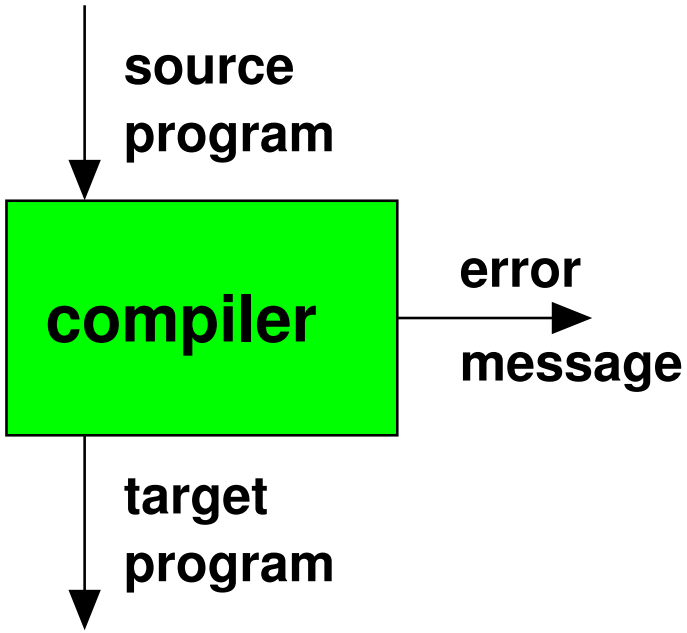
- Compiler reads source program.
- Compiler runs an unreasonable amount of time.
- **Compiler emits target program.**

Compiler: The User's View



- Compiler reads source program.
- Compiler runs an unreasonable amount of time.
- **Compiler emits target program.**
- User is happy.

Compiler: The User's View



- Compiler reads source program.
- Compiler runs an unreasonable amount of time.
- **Compiler emits target program.**
- User is happy.
- Until (s)he sees the (wrong) results.....

Where Do We Go From Here ?

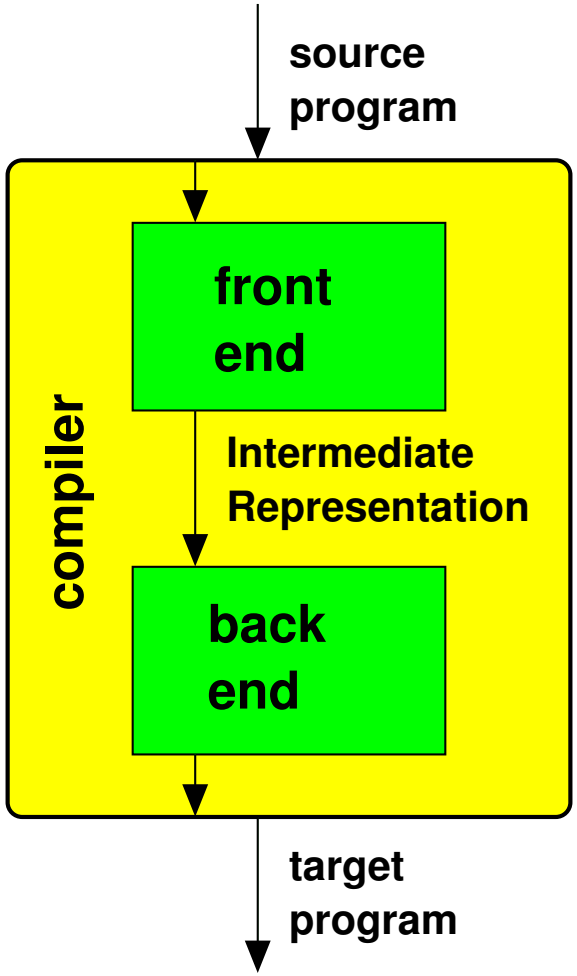
But that's history for you now !!

Where Do We Go From Here ?

But that's history for you now !!

Now, we're going inside a compiler !!

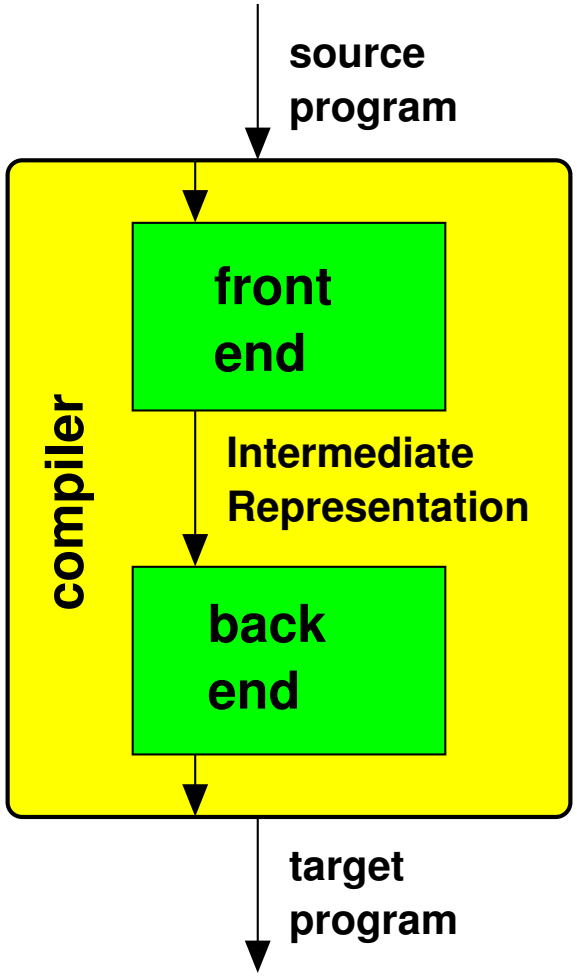
Basic Compiler Structure



Front end:

- Analysis of source program
- Error detection
- Conversion of textual representation into intermediate representation (IR) also known as abstract syntax tree (AST)
- **Dominated by source language**

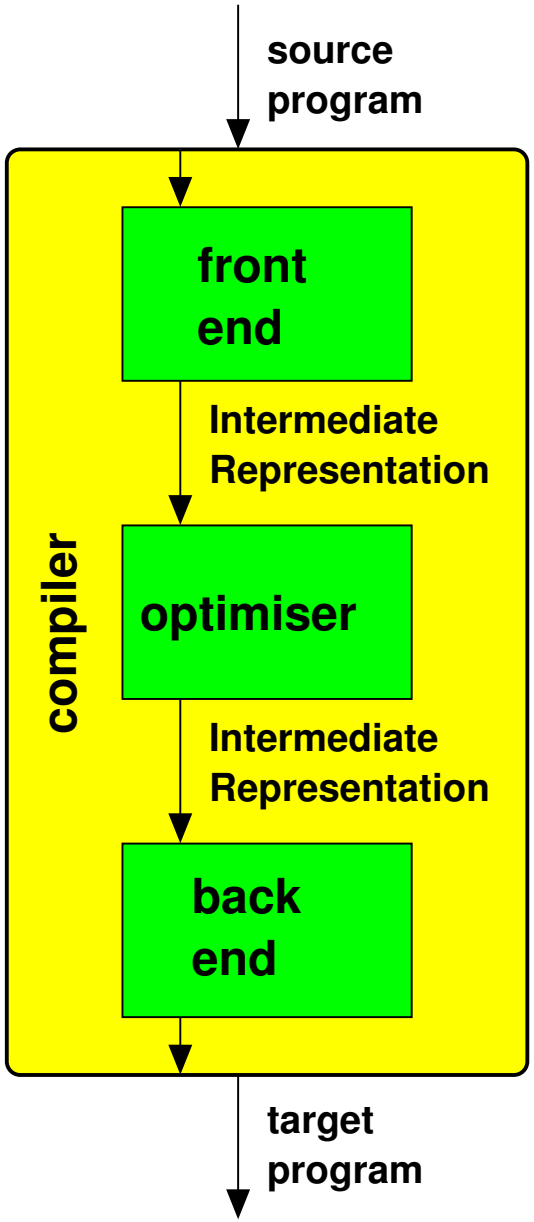
Basic Compiler Structure



Back end:

- Synthesis of target program
- Conversion of intermediate representation into textual representation
- Meeting constraints of target language
- **Dominated by target language**

Advanced Compiler Structure



Front end:

- Analysis of source program

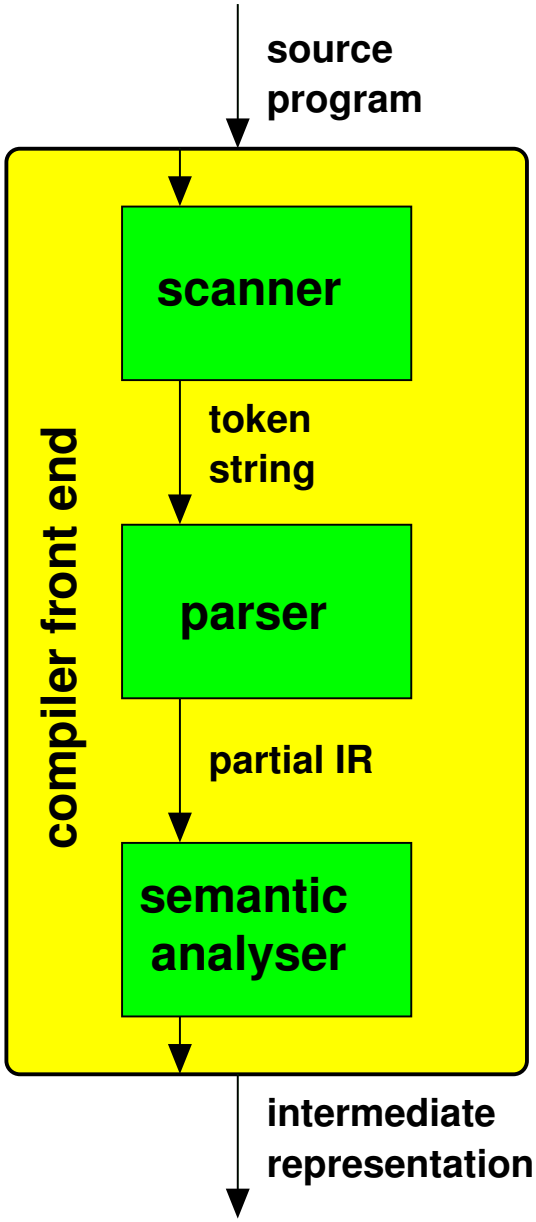
Optimiser (Middle end):

- Code transformation within intermediate representation
- Non-functional motivation:
  - execution speed
  - code size
  - power/energy consumption

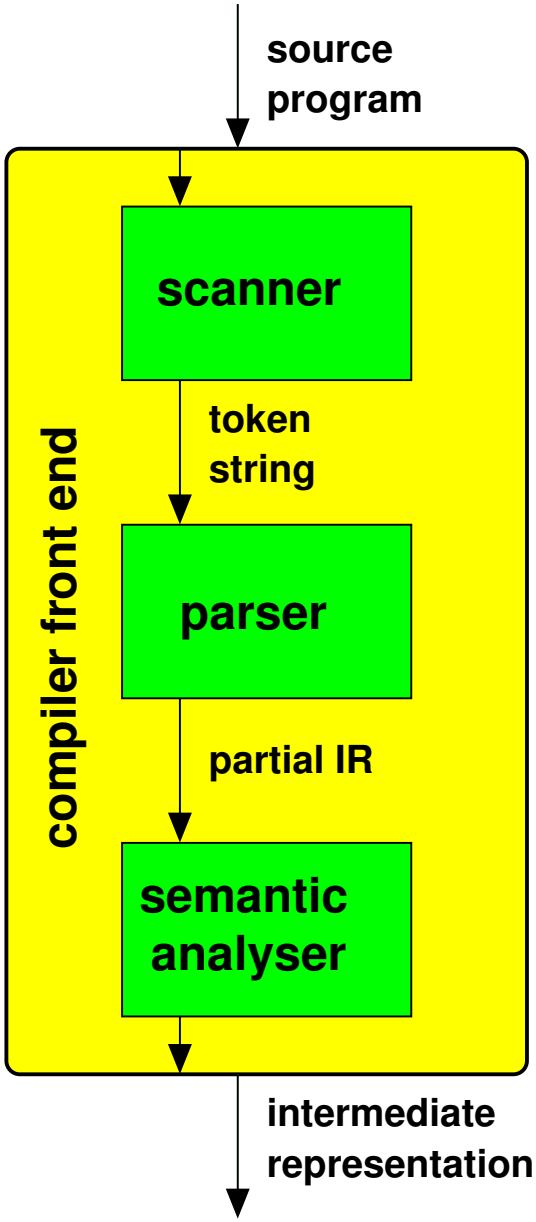
Back end:

- Synthesis of target program

Compiler Frontend



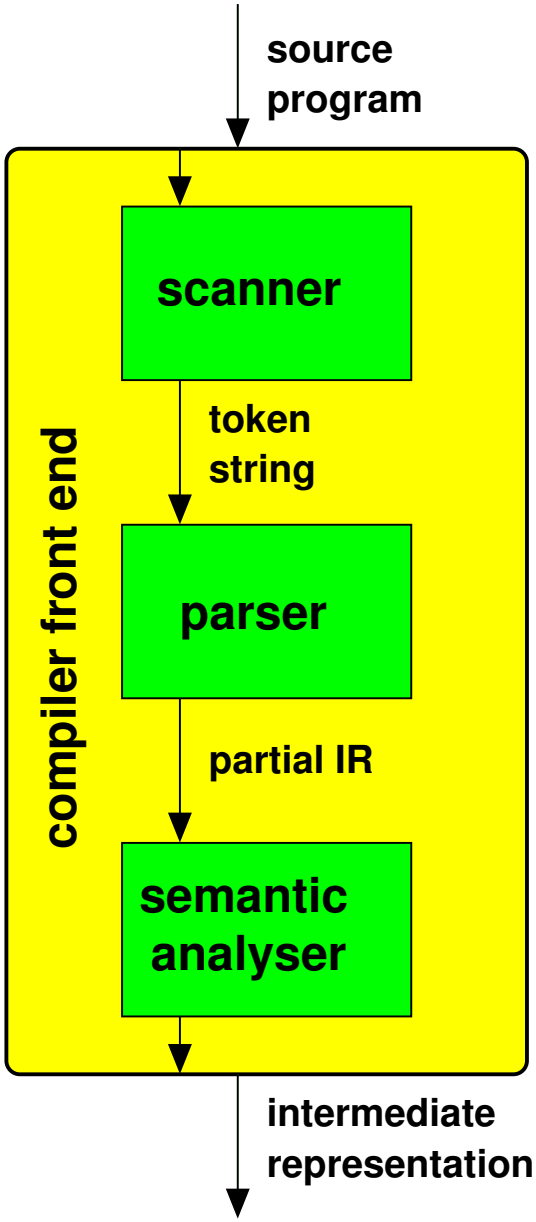
Compiler Frontend



Scanner:

- **Lexical analysis**
- Input: finite stream of characters
- Output: finite stream of **tokens**

Compiler Frontend



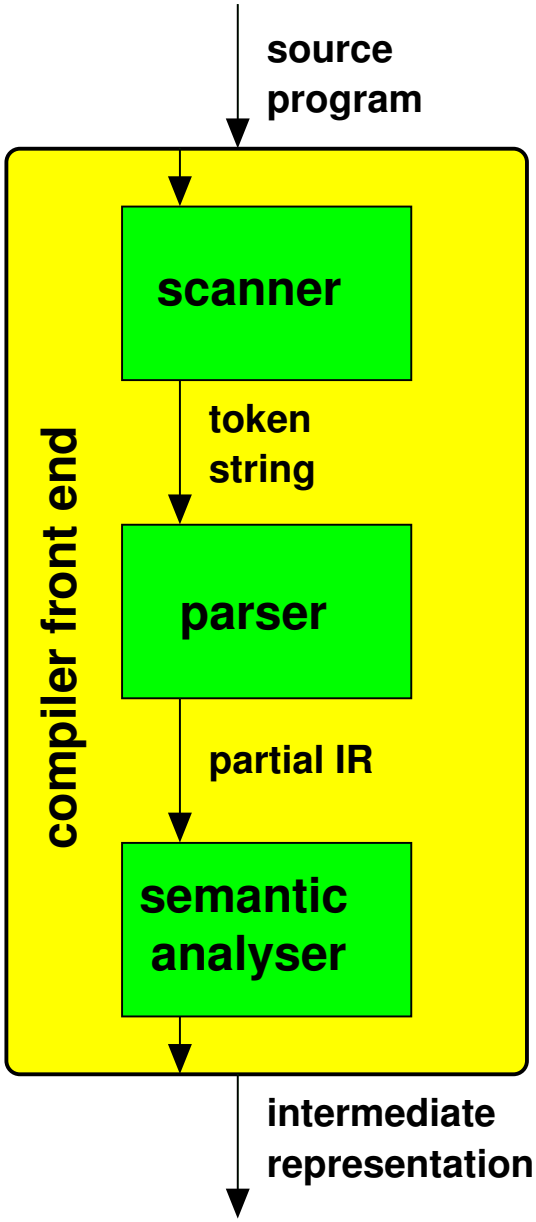
Scanner:

- **Lexical analysis**
- Input: finite stream of characters
- Output: finite stream of **tokens**

Parser:

- **Syntactic analysis**
- Input: finite stream of tokens
- Output: **abstract syntax tree**

Compiler Frontend



Scanner:

- **Lexical analysis**
- Input: finite stream of characters
- Output: finite stream of **tokens**

Parser:

- **Syntactic analysis**
- Input: finite stream of tokens
- Output: **abstract syntax tree**

Semantic analyser:

- **Context analysis** / **Type analysis**
- Input: abstract syntax tree (AST)
- Output: AST with **symbol tables** called **intermediate representation (IR)**

Lexical Analysis Reloaded

Scanner:

- Input: finite stream of characters

Lexical Analysis Reloaded

Scanner:

- Input: finite stream of characters
- Output: finite stream of tokens
  - key words
  - identifiers
  - constants
  - symbols: parentheses, operators, ...

Lexical Analysis Reloaded

Scanner:

- Input: finite stream of characters
- Output: finite stream of tokens
  - key words
  - identifiers
  - constants
  - symbols: parentheses, operators, ...
- Some tokens carry values, others not

Lexical Analysis Reloaded

Scanner:

- Input: finite stream of characters
- Output: finite stream of tokens
  - key words
  - identifiers
  - constants
  - symbols: parentheses, operators, ...
- Some tokens carry values, others not
- Elimination of whitespace (tabs, space, newlines, ...)

Lexical Analysis Reloaded

Scanner:

- Input: finite stream of characters
- Output: finite stream of tokens
  - key words
  - identifiers
  - constants
  - symbols: parentheses, operators, ...
- Some tokens carry values, others not
- Elimination of whitespace (tabs, space, newlines, ...)
- Elimination of comments

Lexical Analysis Reloaded

Scanner:

- Input: finite stream of characters
- Output: finite stream of tokens
  - key words
  - identifiers
  - constants
  - symbols: parentheses, operators, ...
- Some tokens carry values, others not
- Elimination of whitespace (tabs, space, newlines, ...)
- Elimination of comments
- But: tokens associated with lexical information (line/column numbers) for error messages

Lexical Analysis Reloaded

Scanner:

- Input: finite stream of characters
- Output: finite stream of tokens
  - key words
  - identifiers
  - constants
  - symbols: parentheses, operators, ...
- Some tokens carry values, others not
- Elimination of whitespace (tabs, space, newlines, ...)
- Elimination of comments
- But: tokens associated with lexical information (line/column numbers) for error messages
- No errors detected at this stage, unless a specific symbol is disallowed

Scanner Example

Running example:

```
x␣=␣x␣+␣2␣*␣k␣*␣(yy-4);
```

After scanning:

```
ID('x')
ASSIGN
ID('x')
ADD
NUM(2)
MUL
ID('k')
MUL
BRACKET_L
ID('yy')
SUB
NUM(4)
BRACKET_R
SEMICOLON
```

Syntactic Analysis Reloaded

Parser:

- Input: finite stream of tokens

Syntactic Analysis Reloaded

Parser:

- Input: finite stream of tokens
- Output: abstract syntax tree (AST)

Syntactic Analysis Reloaded

Parser:

- Input: finite stream of tokens
- Output: abstract syntax tree (AST)
- Tree structure exposed explicitly

Syntactic Analysis Reloaded

Parser:

- Input: finite stream of tokens
- Output: abstract syntax tree (AST)
- Tree structure exposed explicitly
- Structure-defining tokens vanish
  - semicolons
  - brackets

Syntactic Analysis Reloaded

Parser:

- Input: finite stream of tokens
- Output: abstract syntax tree (AST)
- Tree structure exposed explicitly
- Structure-defining tokens vanish
  - semicolons
  - brackets
- Operator precedence and associativity resolved

Syntactic Analysis Reloaded

Parser:

- Input: finite stream of tokens
- Output: abstract syntax tree (AST)
- Tree structure exposed explicitly
- Structure-defining tokens vanish
  - semicolons
  - brackets
- Operator precedence and associativity resolved
- Detection of syntactic errors

Syntactic Analysis Reloaded

Parser:

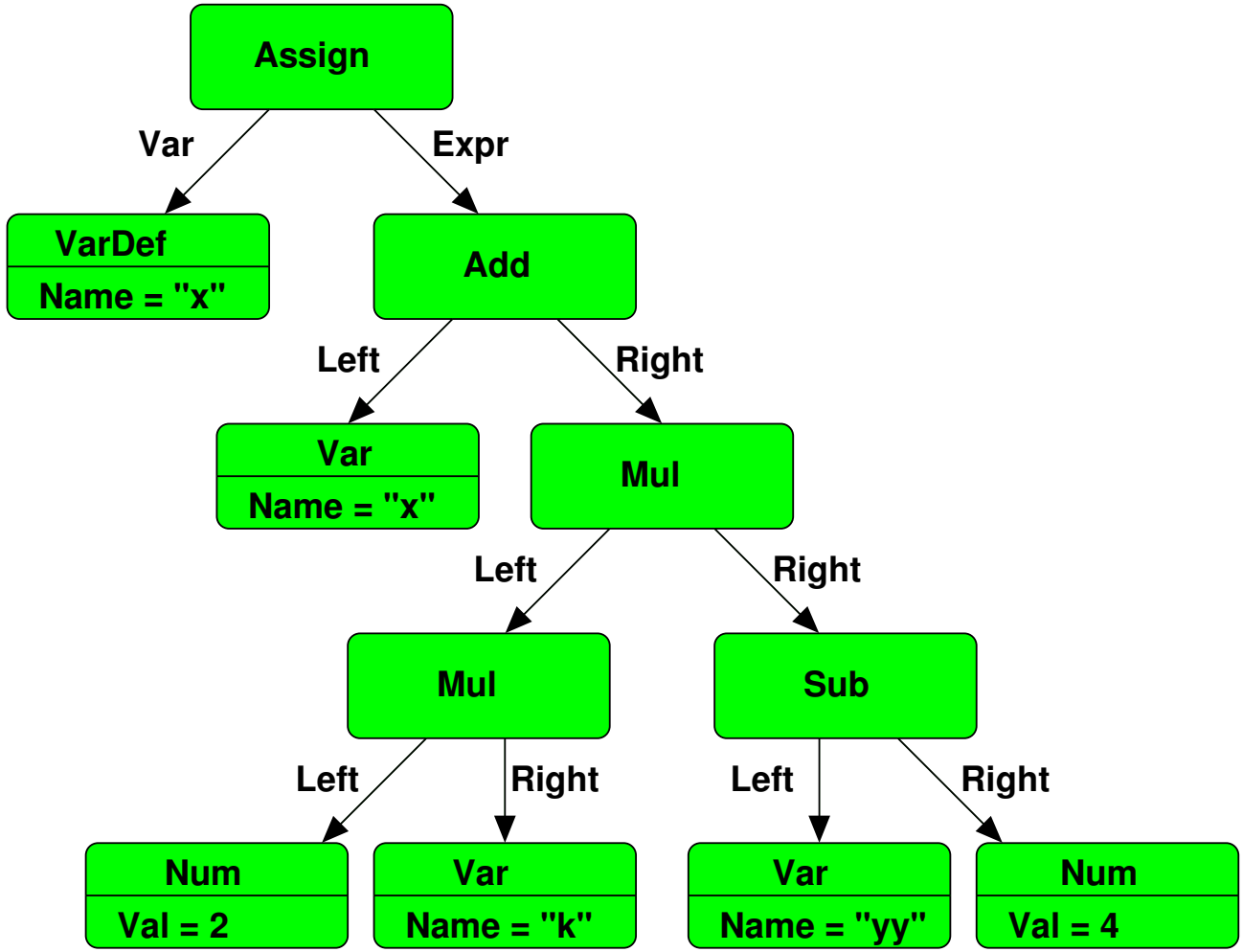
- Input: finite stream of tokens
- Output: abstract syntax tree (AST)
- Tree structure exposed explicitly
- Structure-defining tokens vanish
  - semicolons
  - brackets
- Operator precedence and associativity resolved
- Detection of syntactic errors
- Passed programs called well-formed

Parser Example: x = x + 2 * k * (yy-4);

Running example:

```
ID('x')
ASSIGN
ID('x')
ADD
NUM(2)
MUL
ID('k')
MUL
BRACKET_L
ID('yy')
SUB
NUM(4)
BRACKET_R
SEMICOLON
```

After syntactic analysis:



Semantic Analysis

Part 1: Context Analyser:

- Input: abstract syntax tree (AST)

Semantic Analysis

Part 1: Context Analyser:

- Input: abstract syntax tree (AST)
- Output: abstract syntax tree with symbol tables

Semantic Analysis

Part 1: Context Analyser:

- Input: abstract syntax tree (AST)
- Output: abstract syntax tree with symbol tables
- Variable definitions related to variable declarations

Semantic Analysis

Part 1: Context Analyser:

- Input: abstract syntax tree (AST)
- Output: abstract syntax tree with symbol tables
- Variable definitions related to variable declarations
- Variable uses related to variable declarations

Semantic Analysis

Part 1: Context Analyser:

- Input: abstract syntax tree (AST)
- Output: abstract syntax tree with symbol tables
- Variable definitions related to variable declarations
- Variable uses related to variable declarations
- Function calls related to function definitions

Semantic Analysis

Part 1: Context Analyser:

- Input: abstract syntax tree (AST)
- Output: abstract syntax tree with symbol tables
- Variable definitions related to variable declarations
- Variable uses related to variable declarations
- Function calls related to function definitions
- Type occurrences related to type definitions

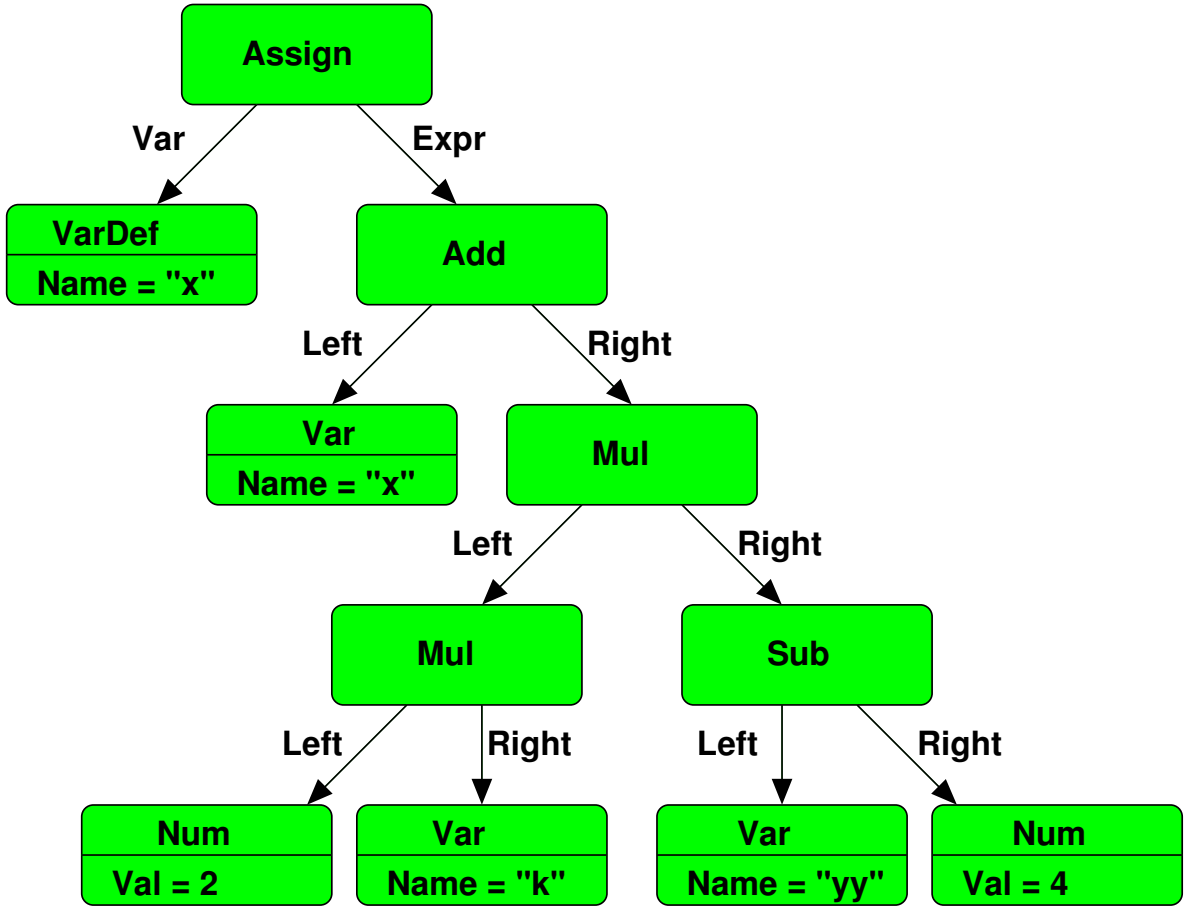
Semantic Analysis

Part 1: Context Analyser:

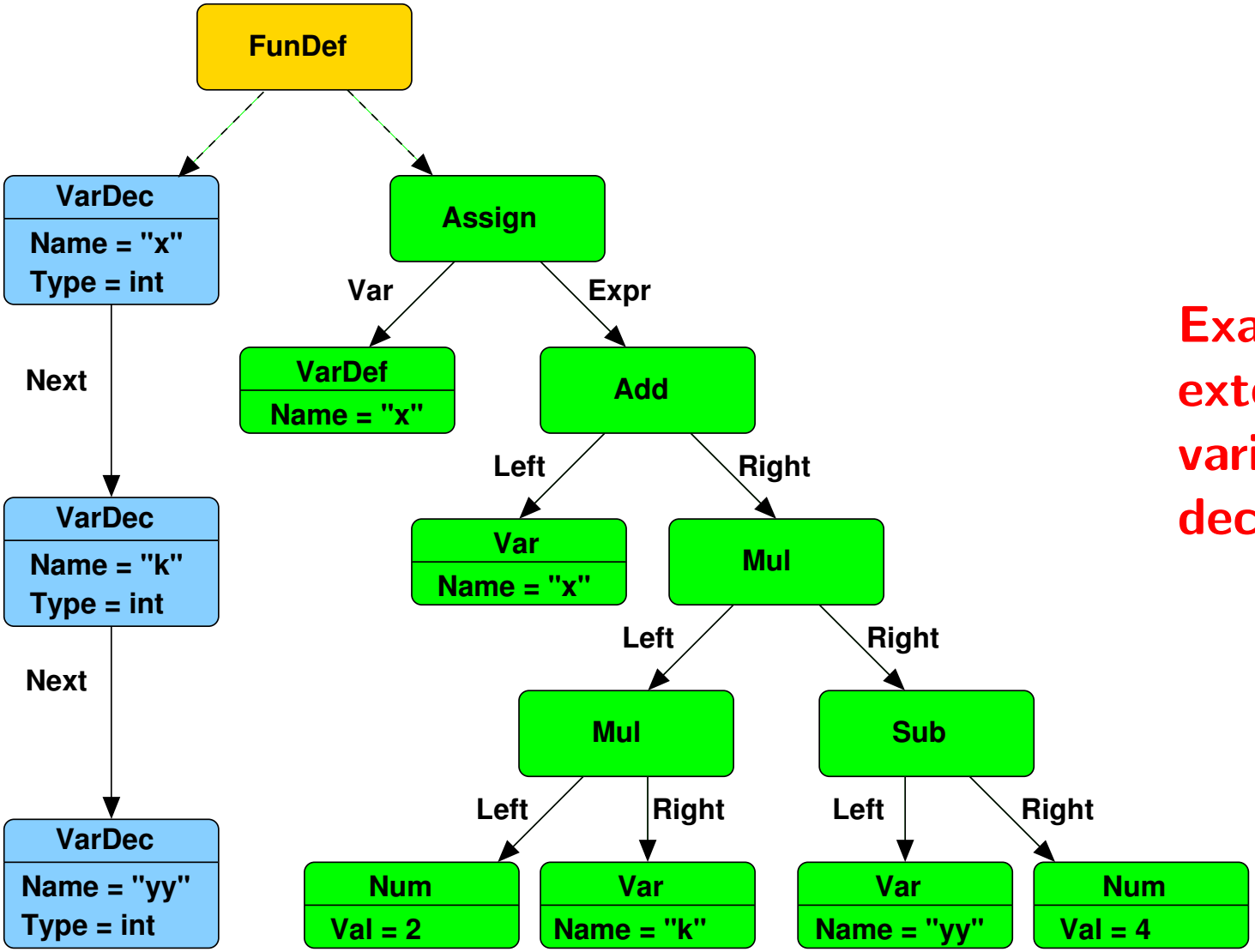
- Input: abstract syntax tree (AST)
- Output: abstract syntax tree with symbol tables
- Variable definitions related to variable declarations
- Variable uses related to variable declarations
- Function calls related to function definitions
- Type occurrences related to type definitions
- In general: identification and resolution of scopes

Context Analyser Example: x = x + 2 * k * (yy-4);

Recap: parser output:



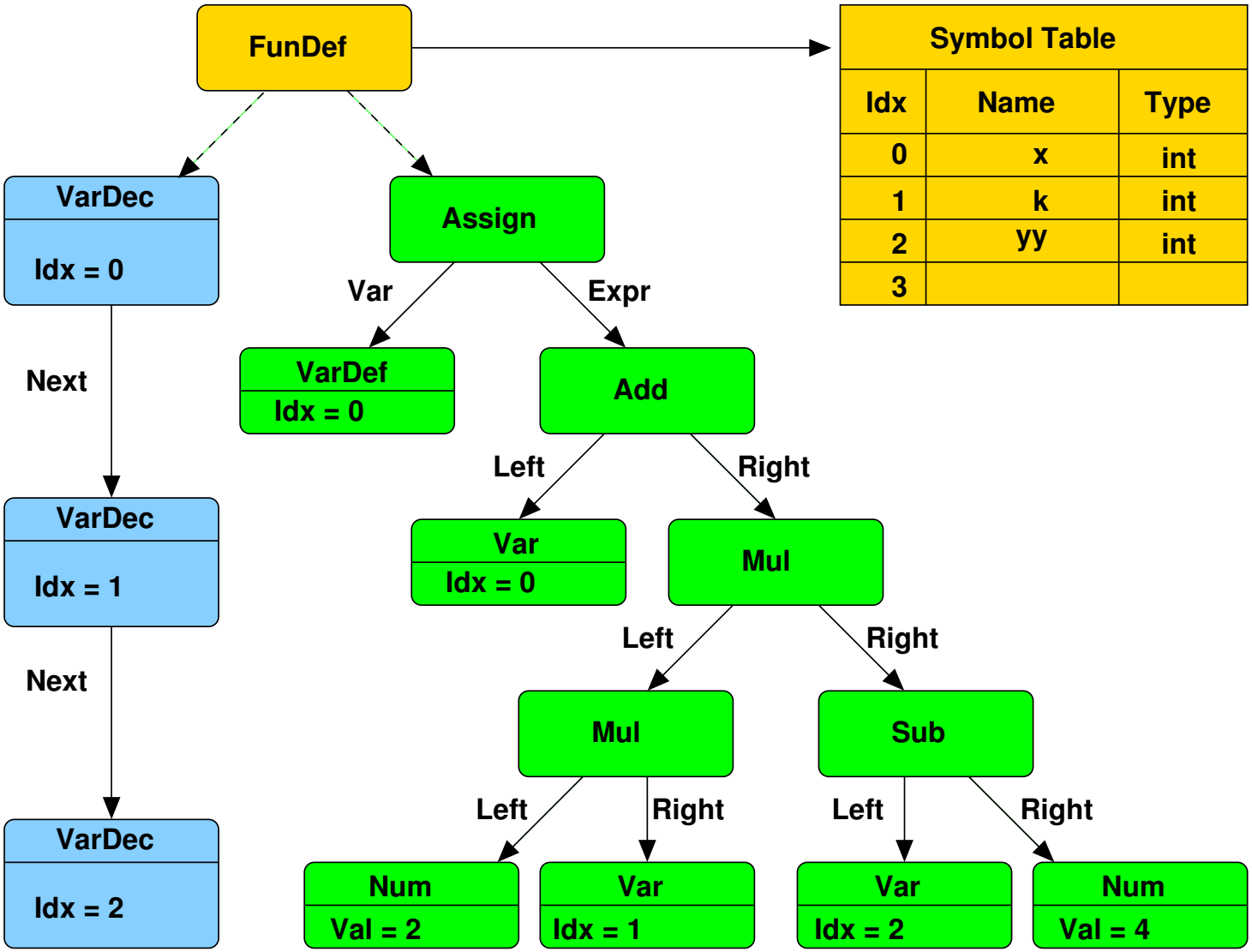
Context Analyser Example: x = x + 2 * k * (yy-4);



Example extended by variable declarations

Context Analyser Example: x = x + 2 * k * (yy-4);

Adding the symbol table:



Semantic Analysis

Part 2: Type Checker:

- Input: abstract syntax tree with symbol tables

Semantic Analysis

Part 2: Type Checker:

- Input: abstract syntax tree with symbol tables
- Output: abstract syntax tree with symbol tables and possibly refined type annotations

Semantic Analysis

Part 2: Type Checker:

- Input: abstract syntax tree with symbol tables
- Output: abstract syntax tree with symbol tables and possibly refined type annotations
- Bottom-up inference of expression types

Semantic Analysis

Part 2: Type Checker:

- Input: abstract syntax tree with symbol tables
- Output: abstract syntax tree with symbol tables and possibly refined type annotations
- Bottom-up inference of expression types
- Expression types match declared variable type upon assignment

Semantic Analysis

Part 2: Type Checker:

- Input: abstract syntax tree with symbol tables
- Output: abstract syntax tree with symbol tables and possibly refined type annotations
- Bottom-up inference of expression types
- Expression types match declared variable type upon assignment
- Function call arguments match function parameter types

Semantic Analysis

Part 2: Type Checker:

- Input: abstract syntax tree with symbol tables
- Output: abstract syntax tree with symbol tables and possibly refined type annotations
- Bottom-up inference of expression types
- Expression types match declared variable type upon assignment
- Function call arguments match function parameter types
- Operator call arguments match built-in operator types

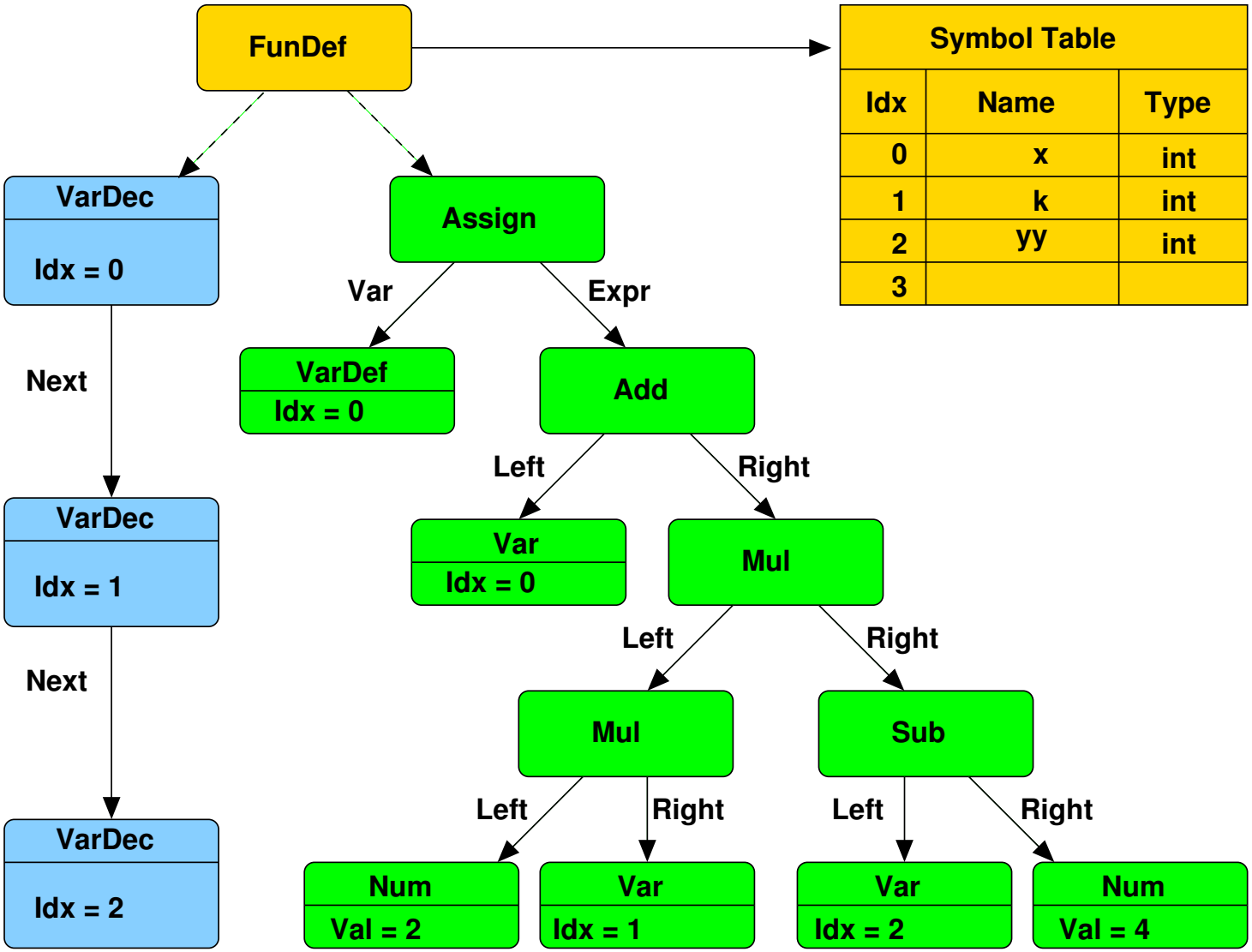
Semantic Analysis

Part 2: Type Checker:

- Input: abstract syntax tree with symbol tables
- Output: abstract syntax tree with symbol tables and possibly refined type annotations
- Bottom-up inference of expression types
- Expression types match declared variable type upon assignment
- Function call arguments match function parameter types
- Operator call arguments match built-in operator types
- Passed programs named well-typed

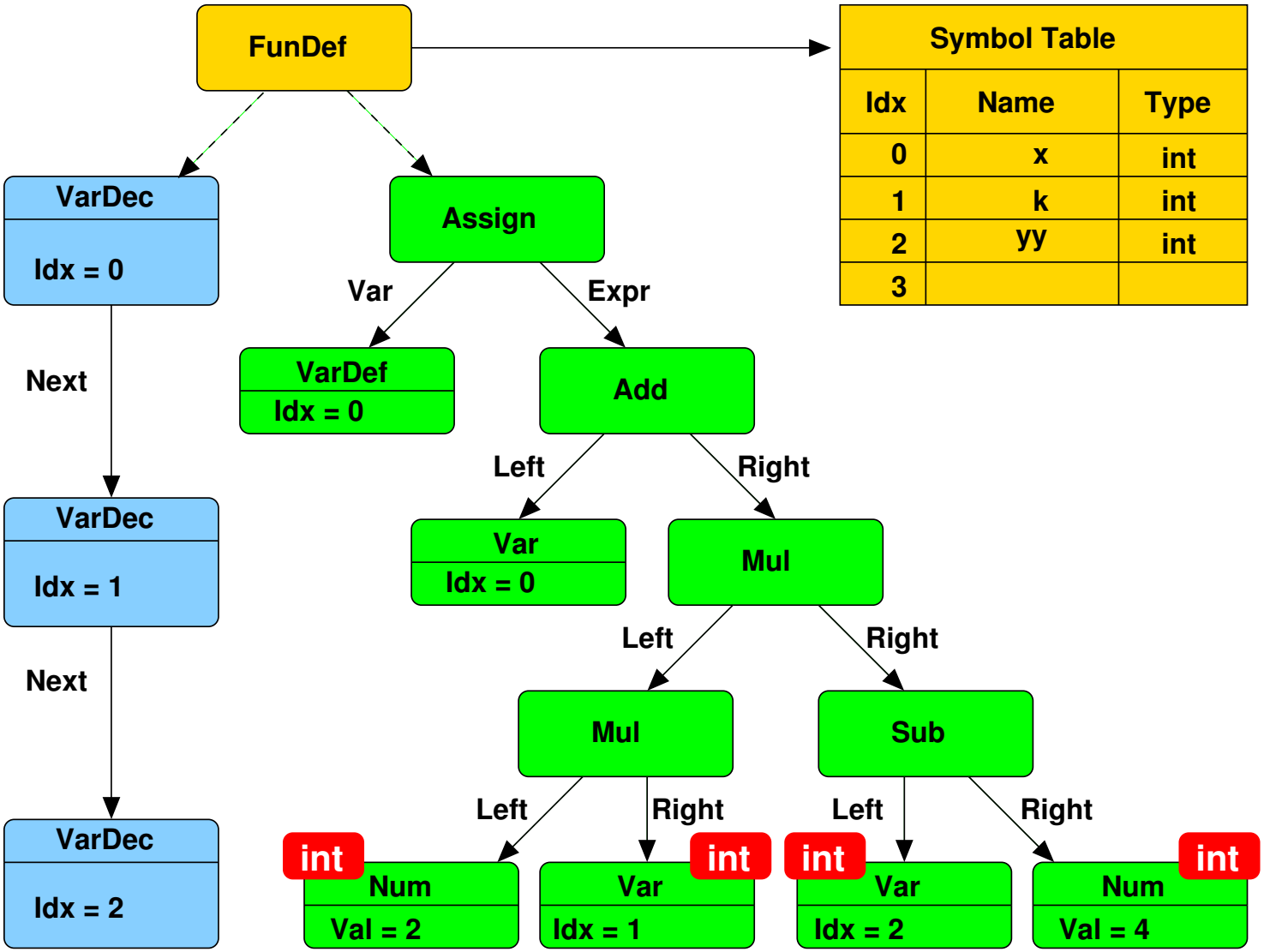
Type Checker Example: x = x + 2 * k * (yy-4);

Recap: context analyser output:



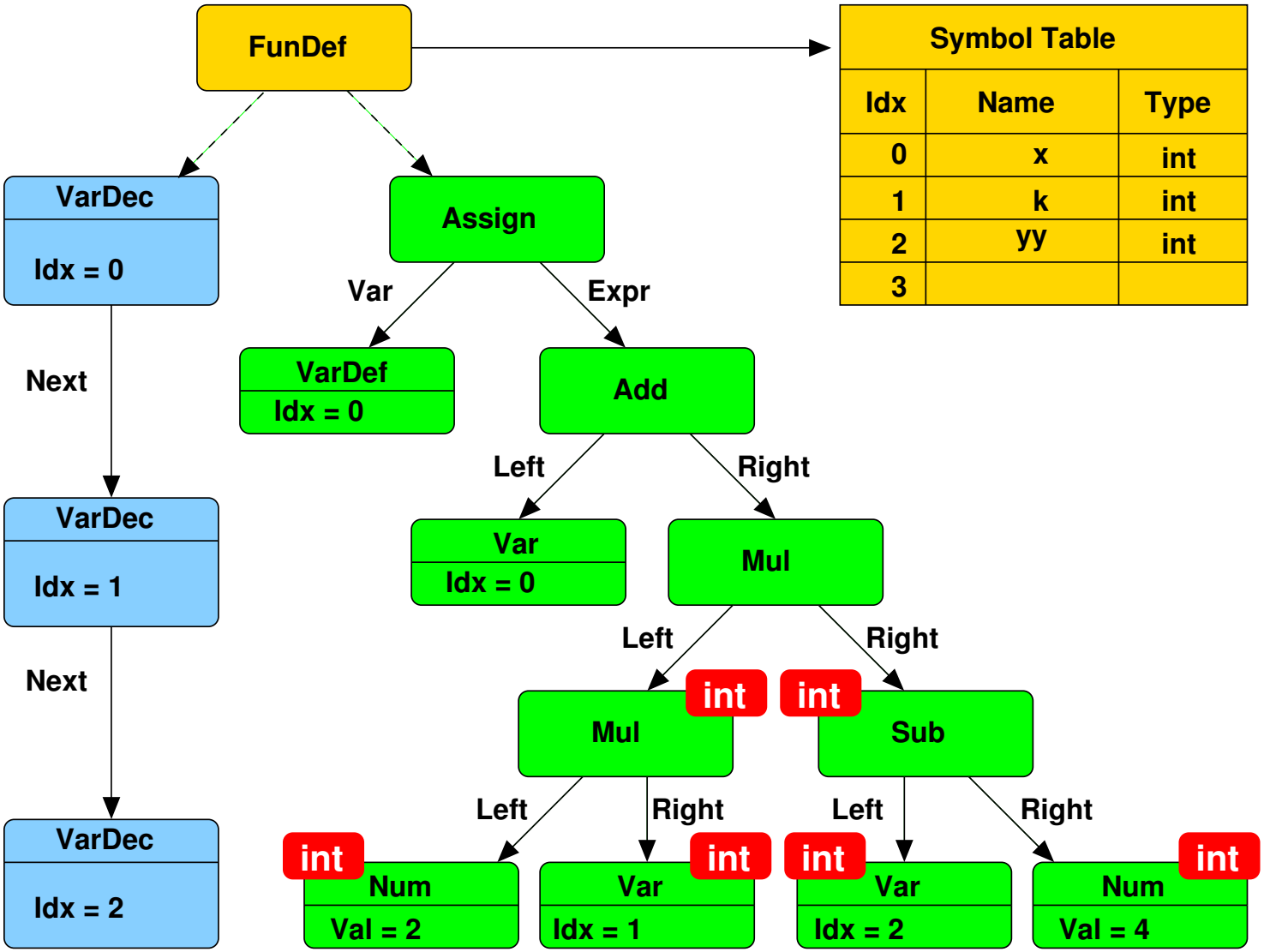
Type Checker Example: x = x + 2 * k * (yy-4);

Inferring expression types bottom-up:



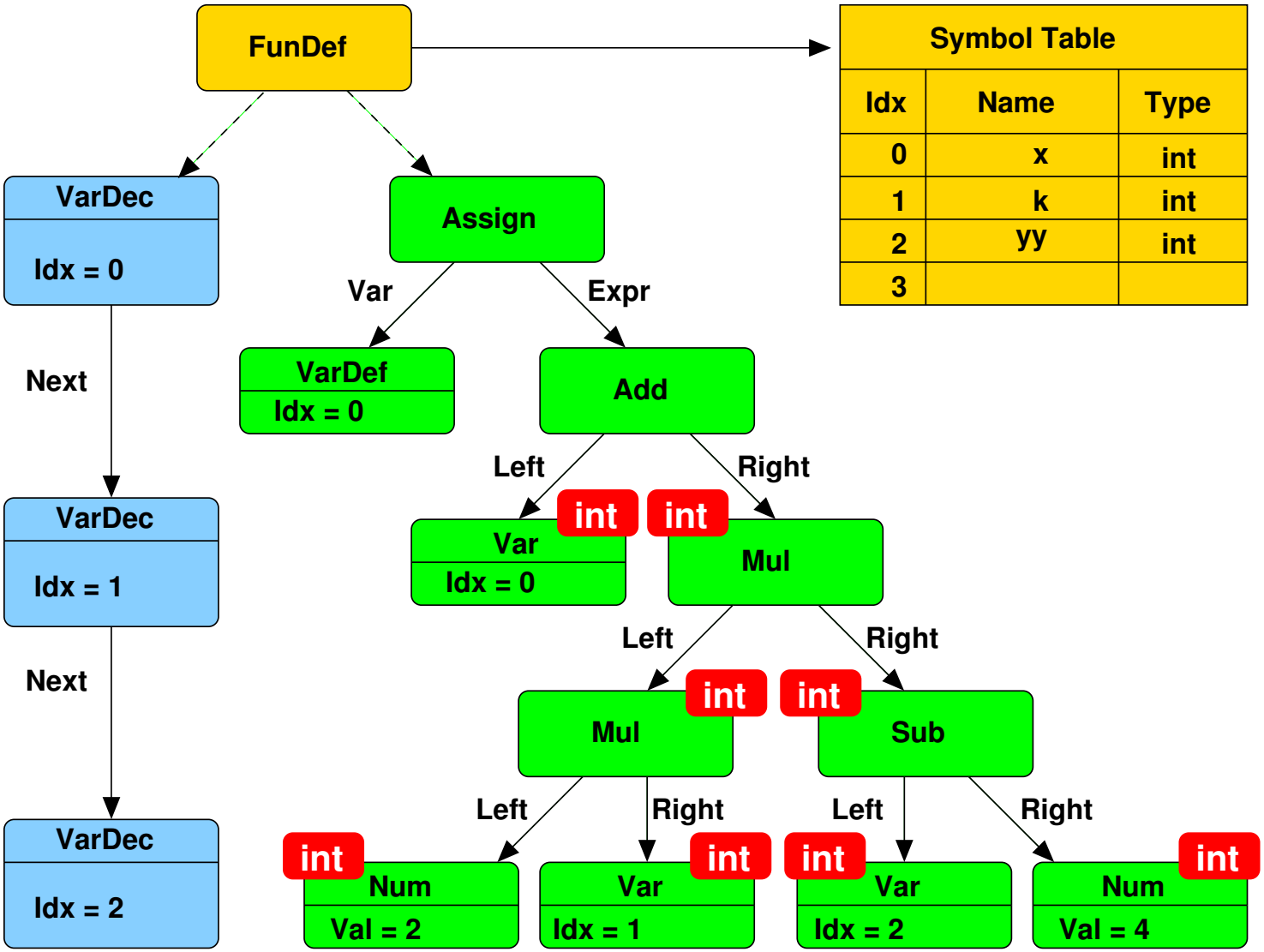
Type Checker Example: x = x + 2 * k * (yy-4);

Inferring expression types bottom-up:



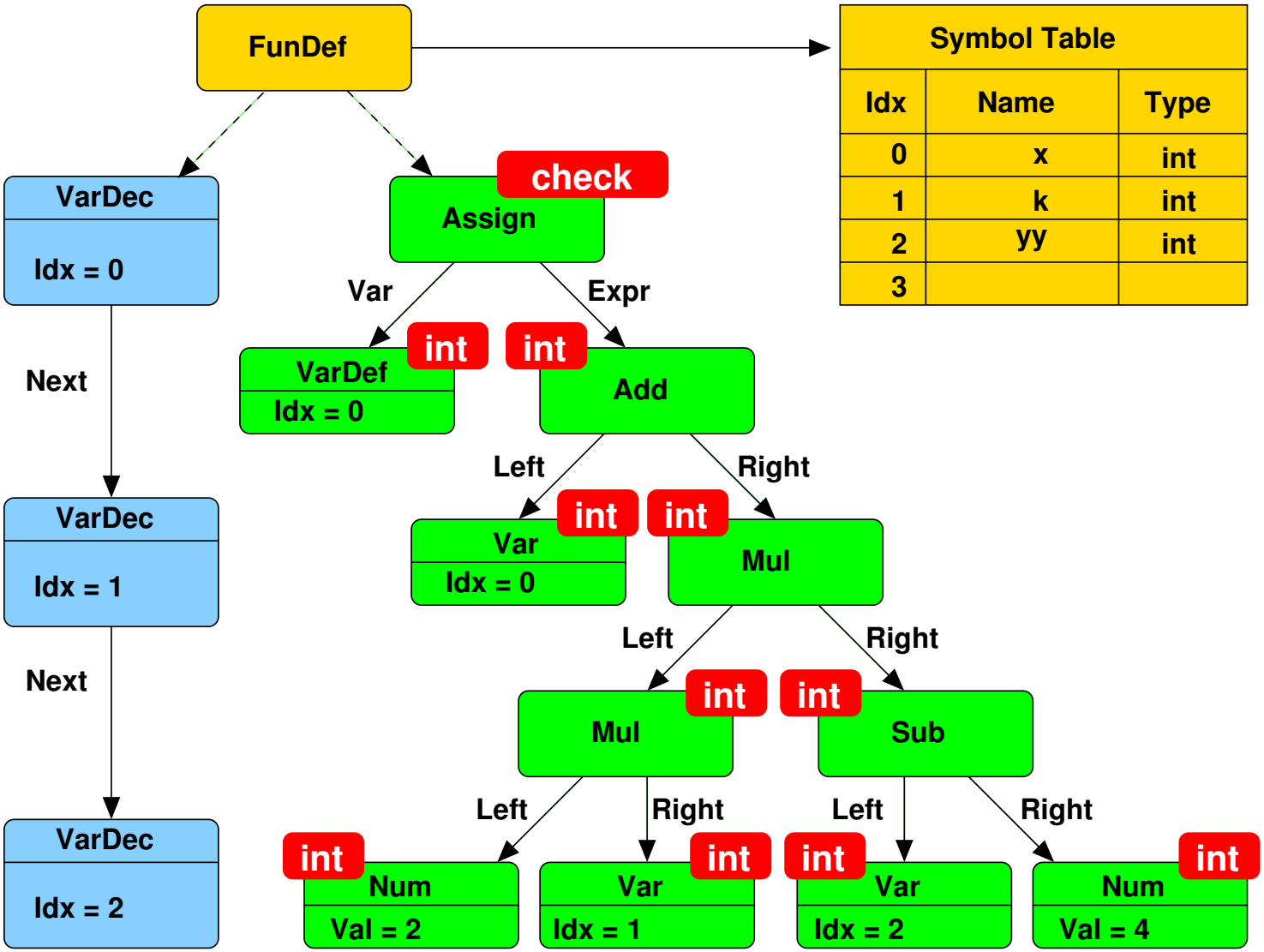
Type Checker Example: x = x + 2 * k * (yy-4);

Inferring expression types bottom-up:

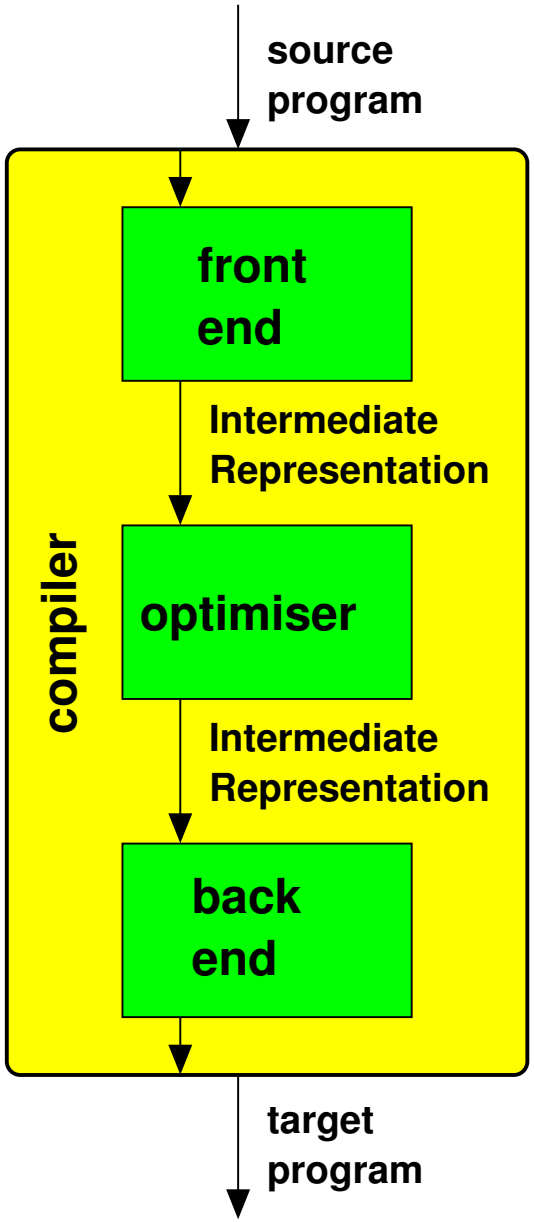


Type Checker Example: x = x + 2 * k * (yy-4);

Checking inferred against declared type:



Advanced Compiler Structure Recap



Front end:

- Analysis of source program

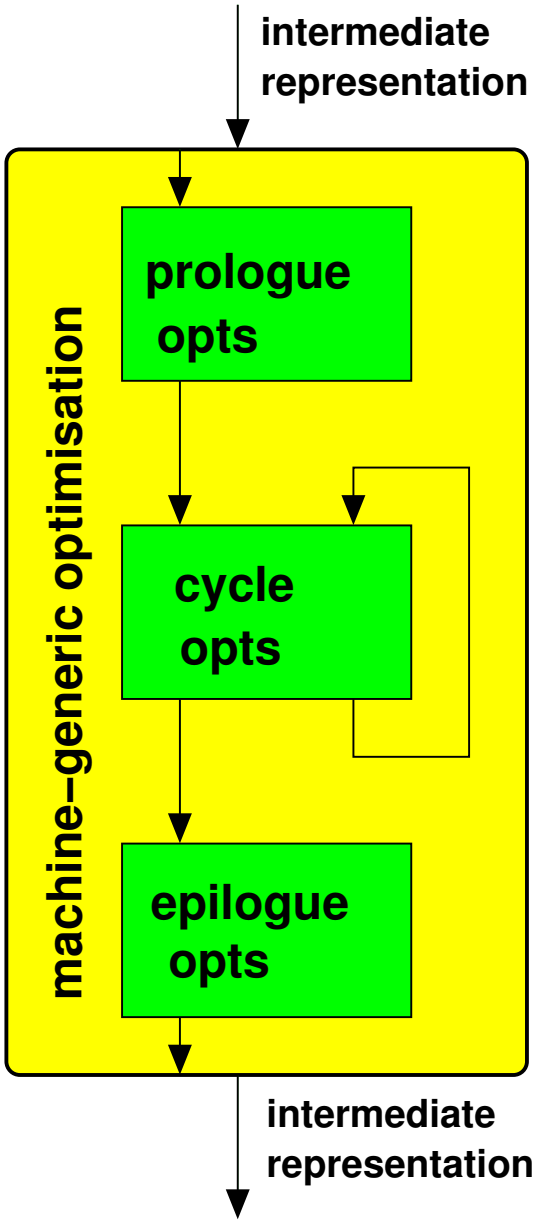
Optimiser (Middle end):

- Code transformation within intermediate representation

Back end:

- Synthesis of target program

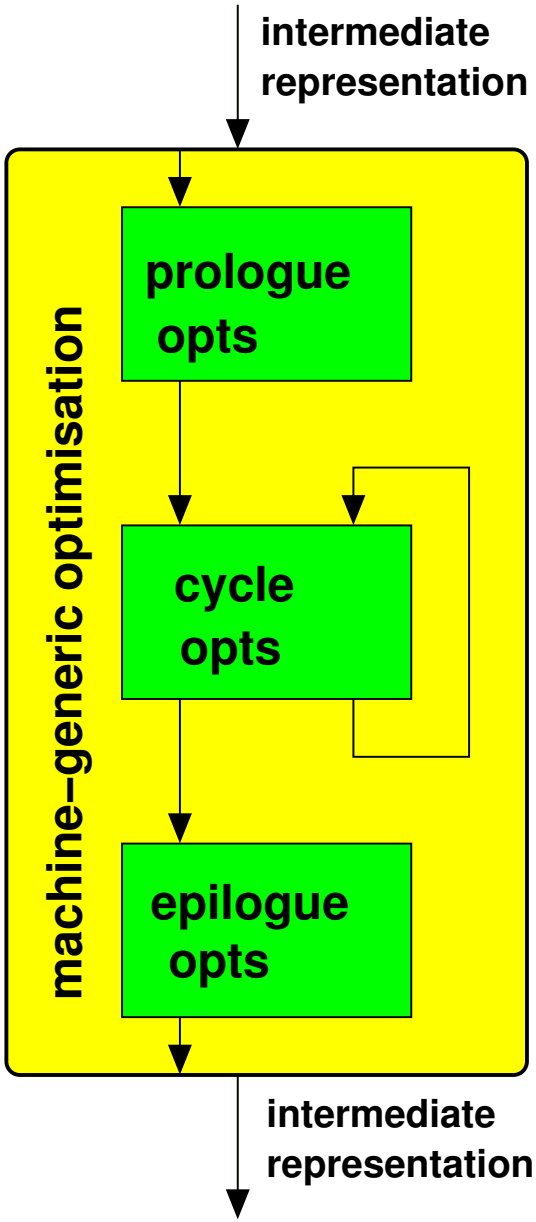
Optimiser



Prologue optimisations:

- function inlining
- ...

Optimiser



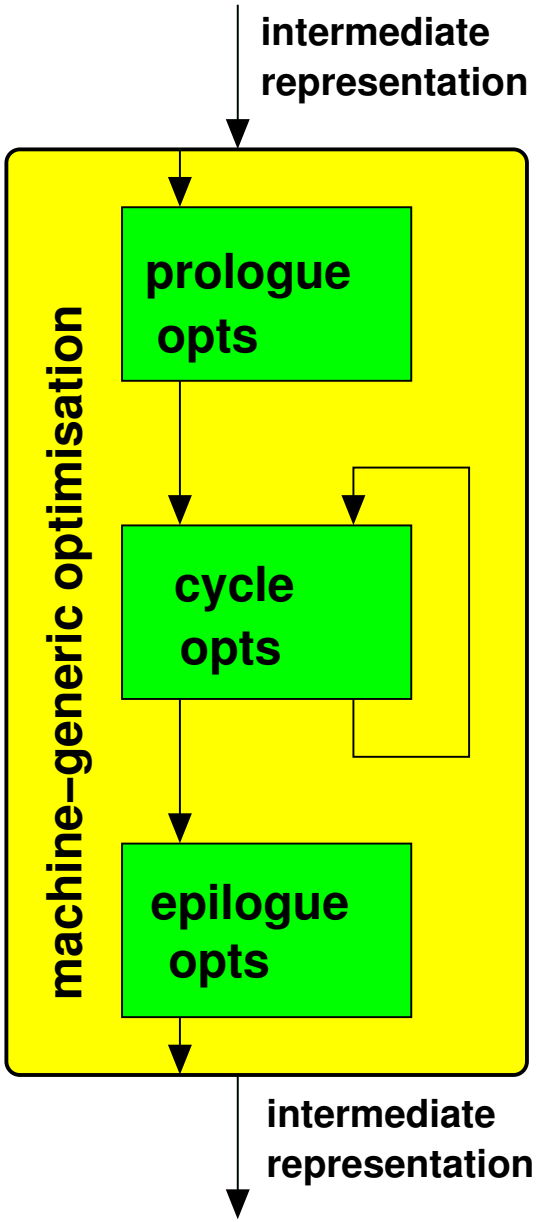
Prologue optimisations:

- function inlining
- ...

Cycle optimisations:

- constant folding
- strength reduction
- common subexpression elimination
- loop unrolling
- ...

Optimiser



Prologue optimisations:

- function inlining
- ...

Cycle optimisations:

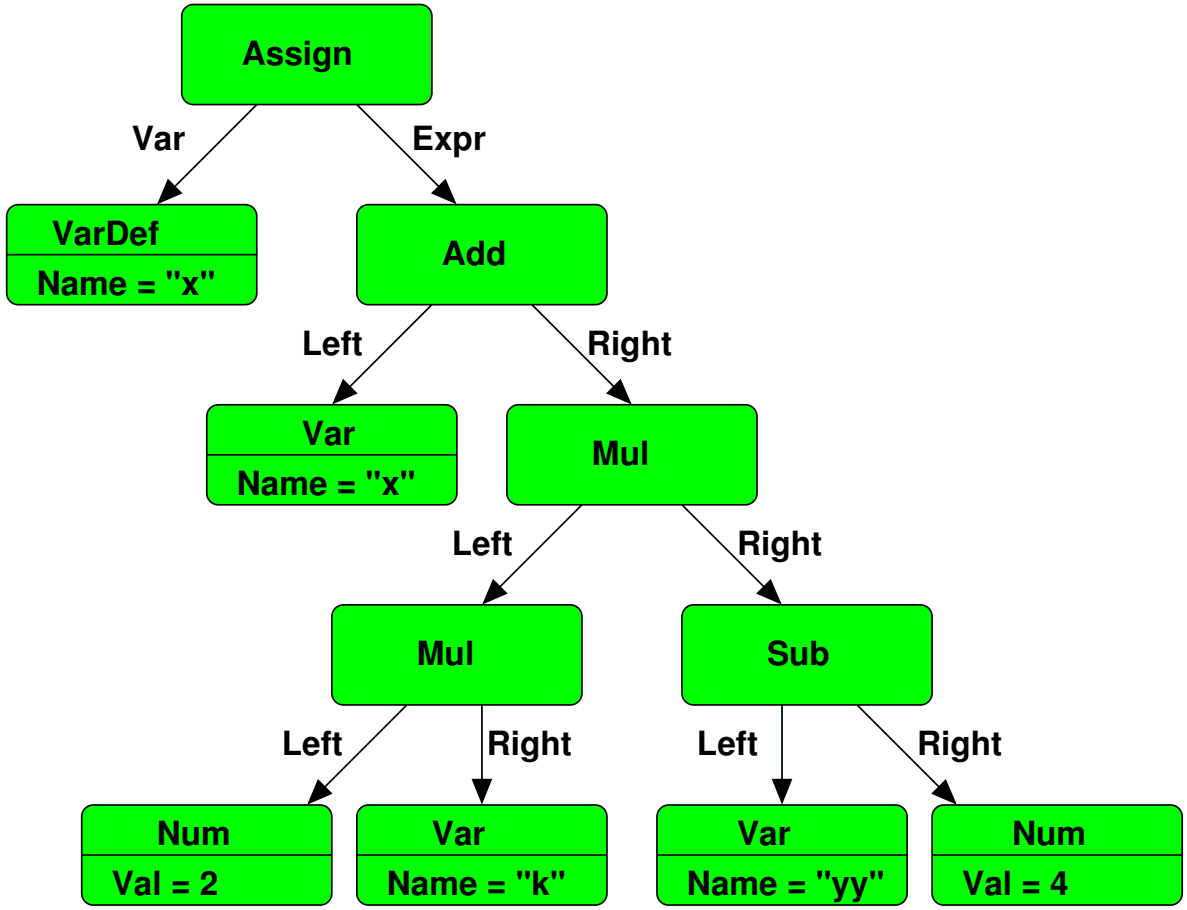
- constant folding
- strength reduction
- common subexpression elimination
- loop unrolling
- ...

Epilogue optimisations:

- dead code removal
- ...

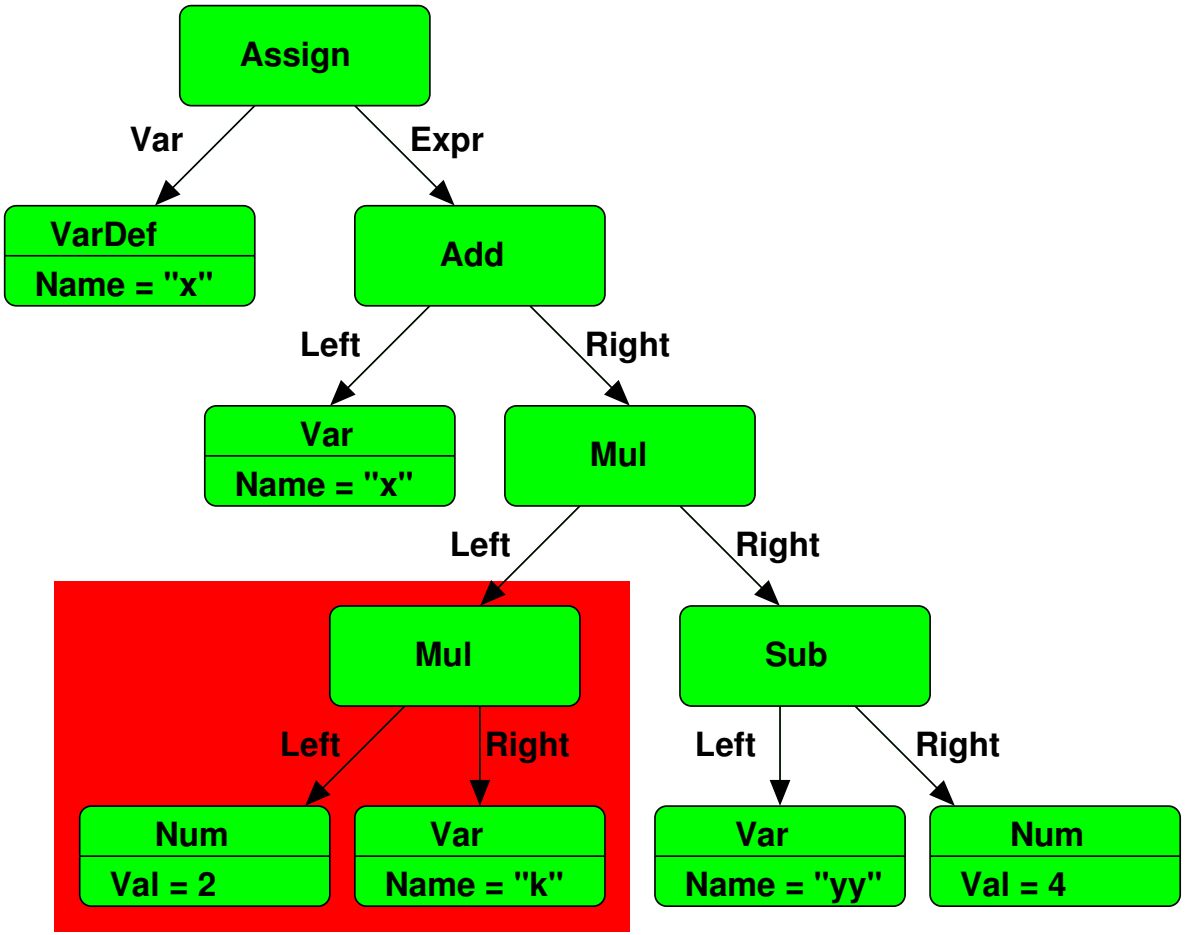
Optimiser Example

Intermediate representation:



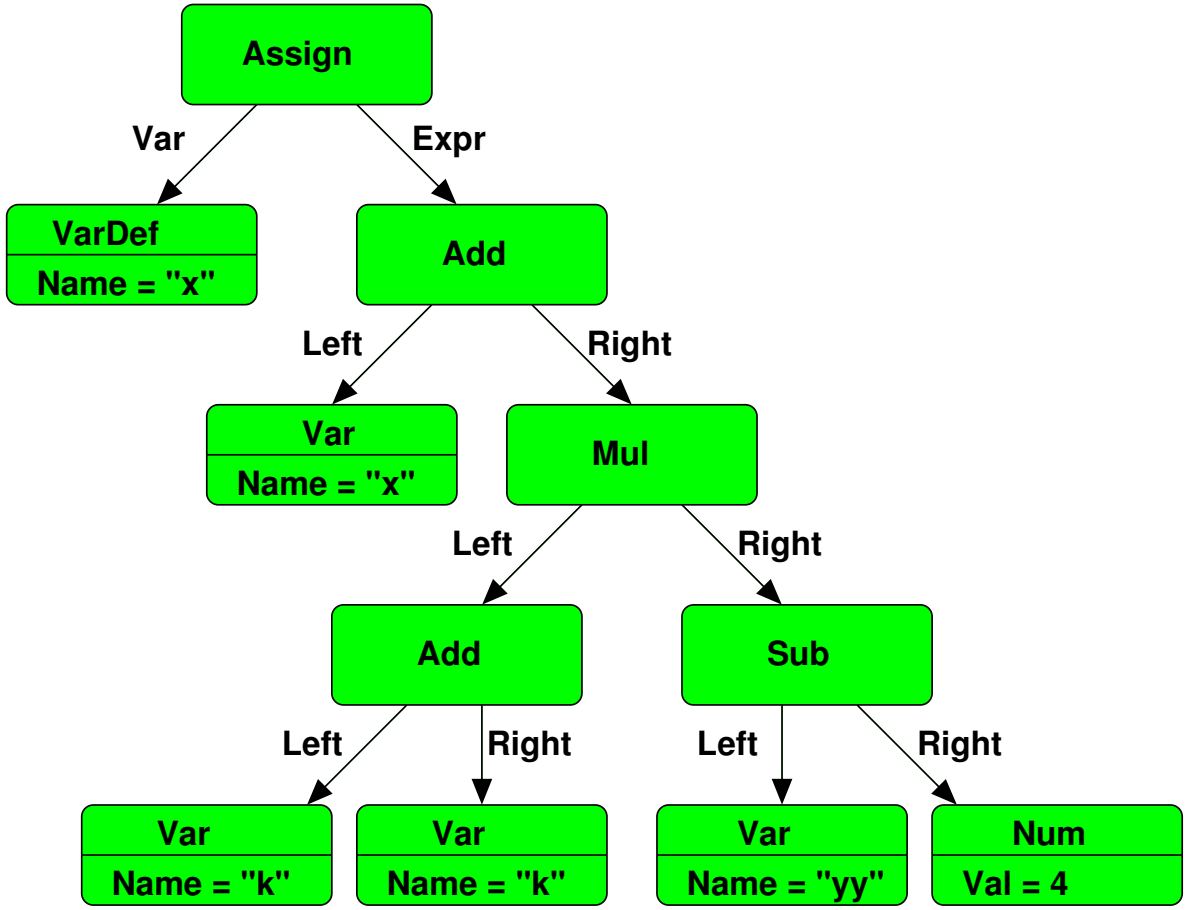
Optimiser Example

Identification of optimisation case:

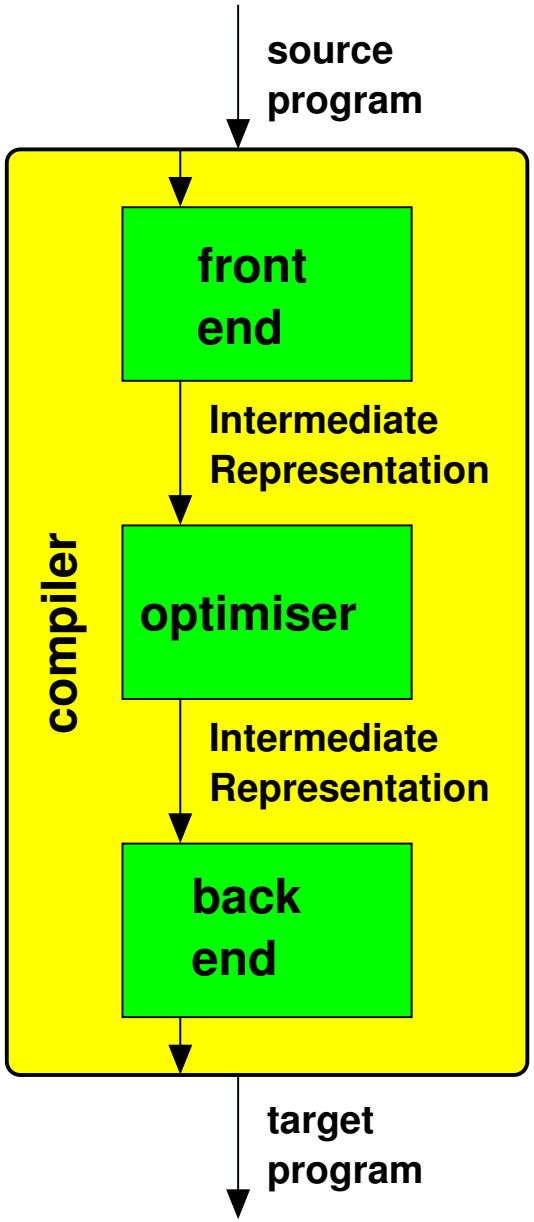


Optimiser Example

Substitution of original code by optimised code:



Advanced Compiler Structure Recap



Front end:

- Analysis of source program

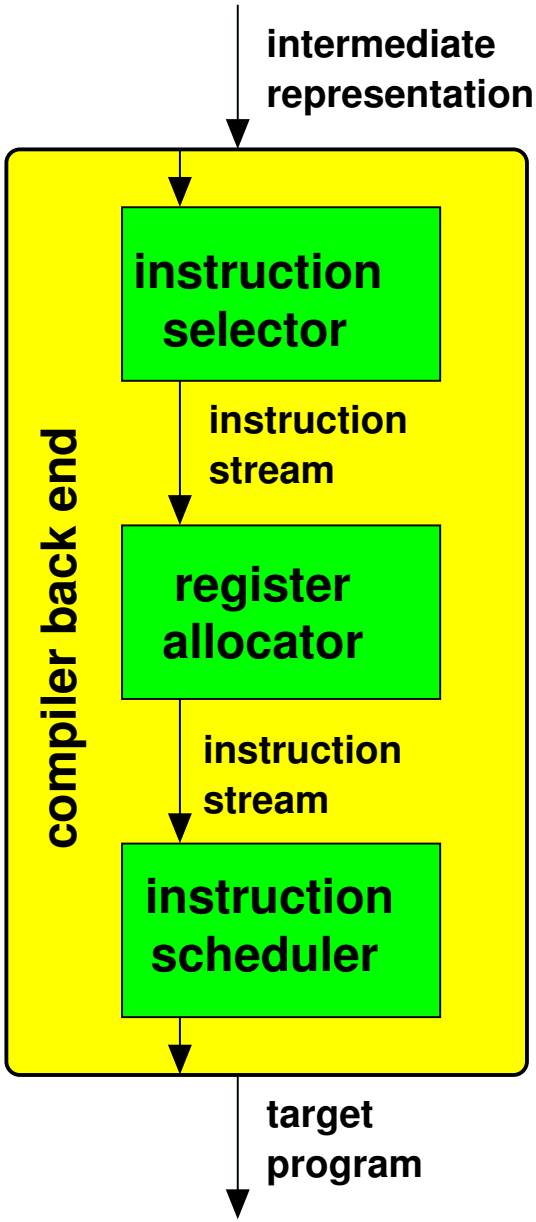
Optimiser (Middle end):

- Code transformation within intermediate representation

Back end:

- Synthesis of target program

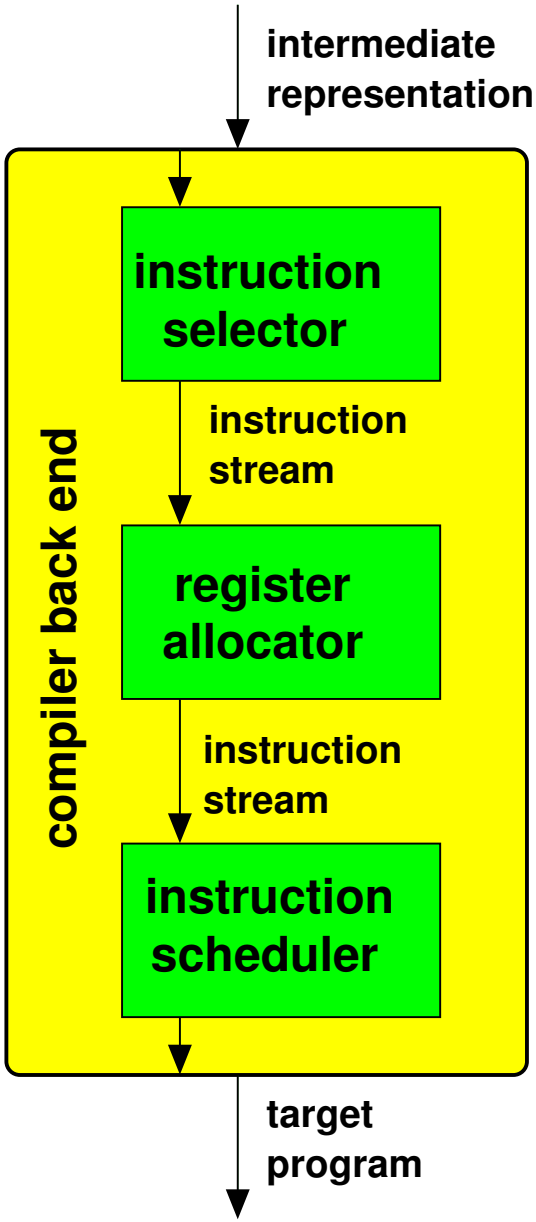
Compiler Backend



Instruction selector:

- Input: abstract syntax tree
- Output: finite stream of assembly instructions
- Unbounded number of registers

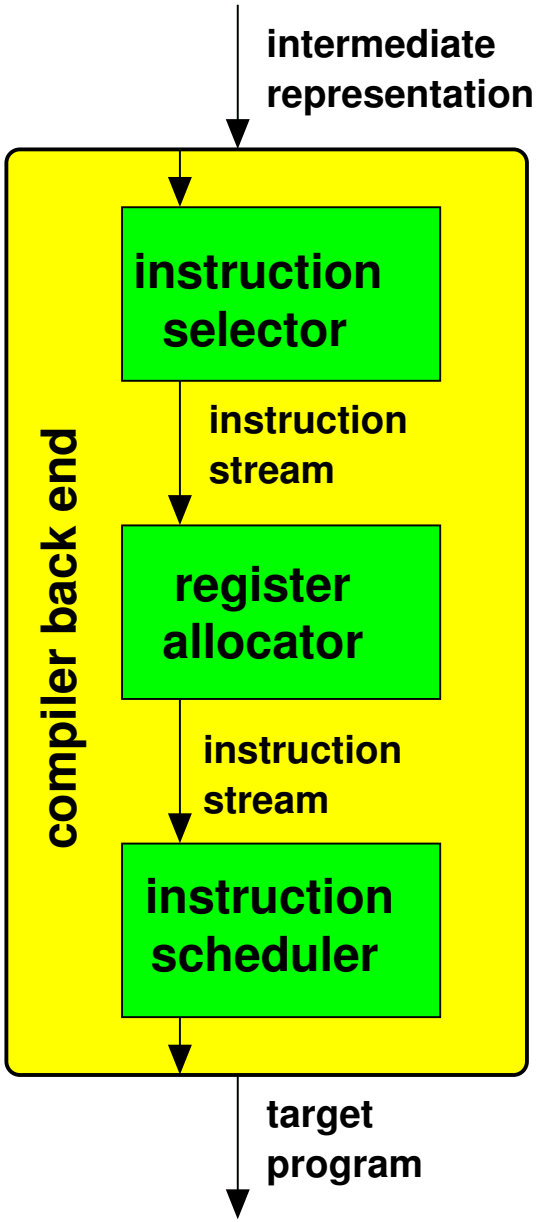
Compiler Backend



Register allocator:

- Input: finite stream of assembly instructions
- Output: finite stream of assembly instructions
- Bounded number of registers used
- Spilling to runtime stack

Compiler Backend



Instruction scheduler:

- Input: finite stream of assembly instructions
- Output: finite stream of assembly instructions
- Instruction reordering
- For superscalar architectures
- Reduce required number of cycles

Instruction Selector

General procedure:

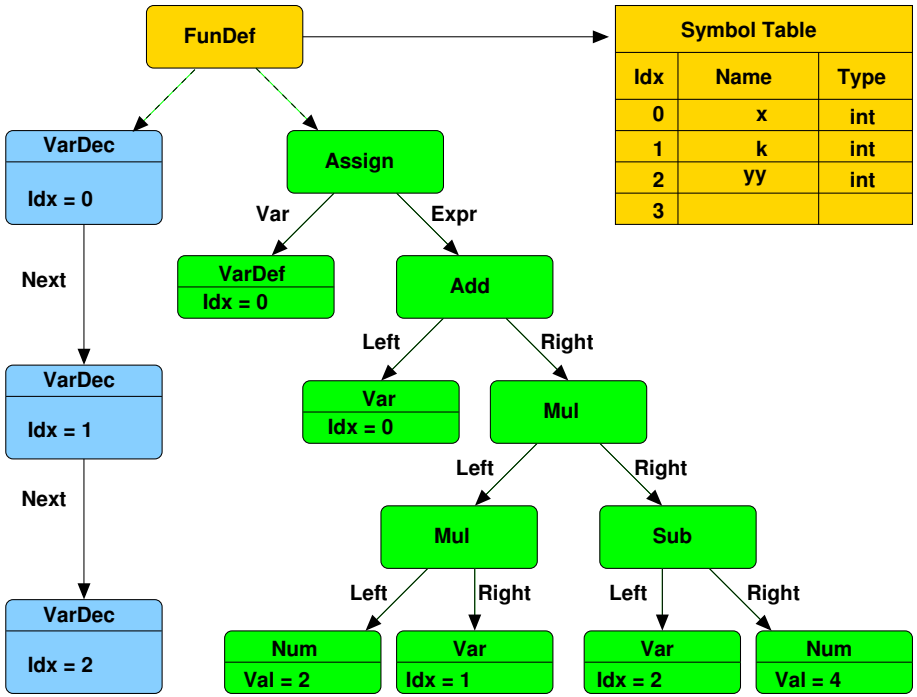
- Input: abstract syntax tree
- Output: finite stream of assembly instructions
- Unbounded number of registers used
- Target-architecture specific
- Leave tree-like intermediate representation
- Re-enter linear representation

Instruction Selector Example

Assumption:

Register $1 points to current stack frame

Example AST:



Algorithm:

- Node defines traversal order
- Assign: right, left
- BinOp: left, right, node

Result:

```
loadAI    $2   <- $1, 0
loadI     $3   <- 2
loadAI    $4   <- $1, 1
mul       $5   <- $3, $4
loadAI    $6   <- $1, 2
loadI     $7   <- 4
sub       $8   <- $6, $7
mul       $9   <- $5, $8
add       $10  <- $2, $9
storeAI   $1, 0 <- $10
```

Register Allocator

General procedure:

- Input: finite stream of assembly instructions
- Output: finite stream of assembly instructions
- Bounded number of registers used
- Spilling to runtime stack where needed
- Target architecture specific constraints established
- Optimal solution is NP-complete
- Heuristics used in practice

Register Allocator

General procedure:

- Input: finite stream of assembly instructions
- Output: finite stream of assembly instructions
- Bounded number of registers used
- Spilling to runtime stack where needed
- Target architecture specific constraints established
- Optimal solution is NP-complete
- Heuristics used in practice

Example heuristics:

- Keep sequence of instructions as generated
- Reuse least numbered register not in use later

Register Allocator Example

Before:

```
loadAI    $2  <- $1, 0
loadI     $3  <- 2
loadAI    $4  <- $1, 1
mul       $5  <- $3, $4
loadAI    $6  <- $1, 2
loadI     $7  <- 4
sub       $8  <- $6, $7
mul       $9  <- $5, $8
add       $10 <- $2, $9
storeAI   $1, 0 <- $10
```

10 registers

After:

```
loadAI    $2  <- $1, 0
loadI     $3  <- 2
loadAI    $4  <- $1, 1
mul       $3  <- $3, $4
loadAI    $4  <- $1, 2
loadI     $5  <- 4
sub       $4  <- $4, $5
mul       $5  <- $3, $4
add       $2  <- $2, $5
storeAI   $1, 0 <- $2
```

5 registers

Instruction Scheduler

General procedure:

- Input: finite stream of assembly instructions
- Output: finite stream of assembly instructions
- Each instruction is associated with a cycle count
- Result will only be available in target register after some cycles
- If target register accessed too early, processor stalls
- Reduce stalling through instruction reordering
- Optimal solution again NP-complete
- Heuristics used in practice
- May lead to increased register usage

Instruction Scheduler Example

Assumptions on target architecture:

- One instruction issued per cycle
- Memory load/store: 5 cycles to completion
- Multiplication: 3 cycles to completion
- All others: 1 cycle to completion

Instruction Scheduler Example

Assumptions:

- Memory load/store: 5 cycles to completion
- Multiplication: 3 cycles to completion
- All others: 1 cycle to completion

Before IS:

```
 1   5 loadAI   $2 <- $1, 0
 2   2 loadI    $3 <- 2
 3   7 loadAI   $4 <- $1, 1
 8  10 mul      $3 <- $3, $4
 9  13 loadAI   $4 <- $1, 2
10  10 loadI    $7 <- 4
14  14 sub      $4 <- $4, $7
15  17 mul      $7 <- $3, $4
18  18 add      $2 <- $2, $7
19  23 storeAI  $1, 0 <- $2
```

23 cycles

Instruction Scheduler Example

Example heuristic:

- Move instruction to earliest possible available issue slot
- Limitation: dependencies
  - data in argument registers not yet available
  - no way out
- Limitation: anti-dependencies
  - result register still in use
  - can be overcome by using additional registers

Instruction Scheduler Example

Assumptions:

- Memory load/store: 5 cycles to completion
- Multiplication: 3 cycles to completion
- All others: 1 cycle to completion

Before IS:

```
 1   5 loadAI   $2 <- $1, 0
 2   2 loadI    $3 <- 2
 3   7 loadAI   $4 <- $1, 1
 8  10 mul      $3 <- $3, $4
 9  13 loadAI   $4 <- $1, 2
10  10 loadI    $7 <- 4
14  14 sub      $4 <- $4, $7
15  17 mul      $7 <- $3, $4
18  18 add      $2 <- $2, $7
19  23 storeAI  $1, 0 <- $2
```

23 cycles

After IS:

```
 1   5 loadAI   $2 <- $1, 0
 2   2 loadI    $3 <- 2
 3   7 loadAI   $4 <- $1, 1
 4   8 loadAI   $5 <- $1, 2
 5   5 loadI    $7 <- 4
 8  10 mul      $3 <- $3, $4
 9   9 sub      $4 <- $5, $7
11  13 mul      $7 <- $3, $4
14  14 add      $2 <- $2, $7
15  19 storeAI  $1, 0 <- $2
```

19 cycles

Instruction Scheduler Example

Assumptions:

- Memory load/store: 5 cycles to completion
- Multiplication: 3 cycles to completion
- All others: 1 cycle to completion

Before IS:

```
 1   5 loadAI   $2 <- $1, 0
 2   2 loadI    $3 <- 2
 3   7 loadAI   $4 <- $1, 1
 8  10 mul      $3 <- $3, $4
 9  13 loadAI   $4 <- $1, 2
10  10 loadI    $7 <- 4
14  14 sub      $4 <- $4, $7
15  17 mul      $7 <- $3, $4
18  18 add      $2 <- $2, $7
19  23 storeAI  $1, 0 <- $2
```

23 cycles

After IS:

```
 1   5 loadAI   $2 <- $1, 0
 2   2 loadI    $3 <- 2
 3   7 loadAI   $4 <- $1, 1
 4   8 loadAI   $5 <- $1, 2
 5   5 loadI    $7 <- 4
 8  10 mul      $3 <- $3, $4
 9   9 sub      $4 <- $5, $7
11  13 mul      $7 <- $3, $4
14  14 add      $2 <- $2, $7
15  19 storeAI  $1, 0 <- $2
```

19 cycles, but 1 more register

Register Allocation vs Instruction Scheduling

Why do register allocation first ?:

- Solve one problem at a time !

Register Allocation vs Instruction Scheduling

Why do register allocation first ?:

- Solve one problem at a time !
- Instruction scheduling is a minimisation problem

Register Allocation vs Instruction Scheduling

Why do register allocation first ?:

- Solve one problem at a time !
- Instruction scheduling is a minimisation problem
- Register allocation is **not** a minimisation problem:
  Using less registers than available brings no advantage

Register Allocation vs Instruction Scheduling

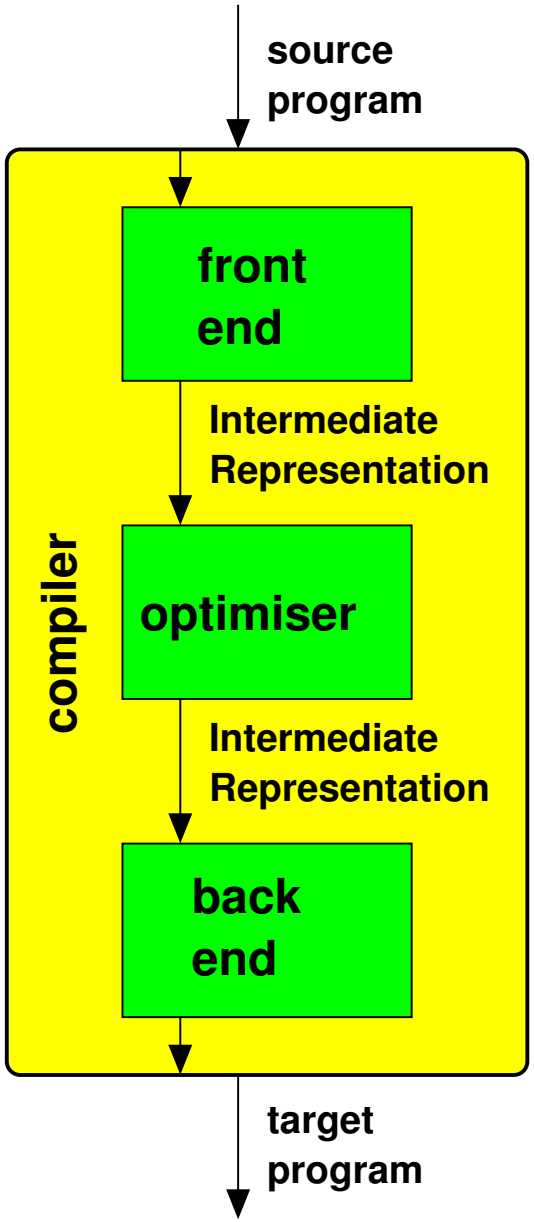
Why do register allocation first ?:

- Solve one problem at a time !
- Instruction scheduling is a minimisation problem
- Register allocation is **not** a minimisation problem:
  Using less registers than available brings no advantage

Solution:

- First, minimize number of registers in use
- Then, use remaining registers for instruction scheduling

Advanced Compiler Structure Recap



Front end:

- Analysis of source program

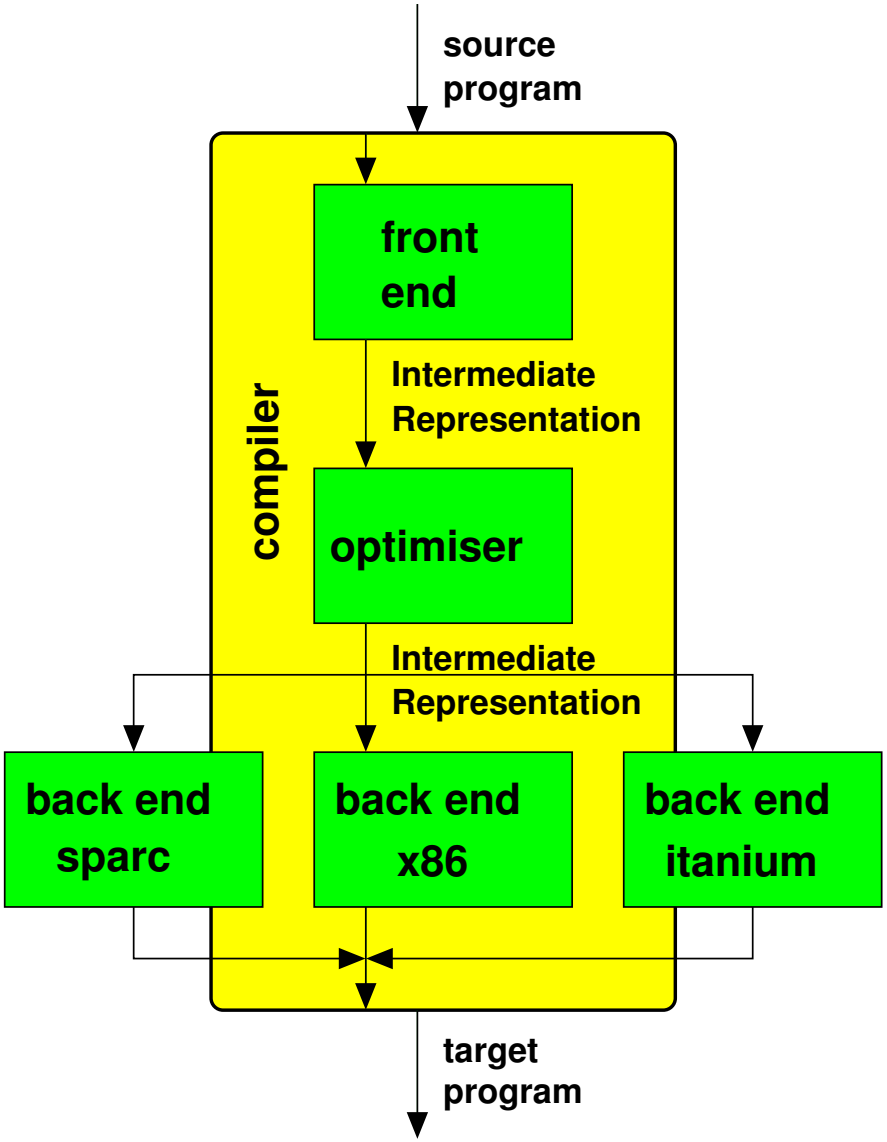
Optimiser (Middle end):

- Code transformation within intermediate representation

Back end:

- Synthesis of target program

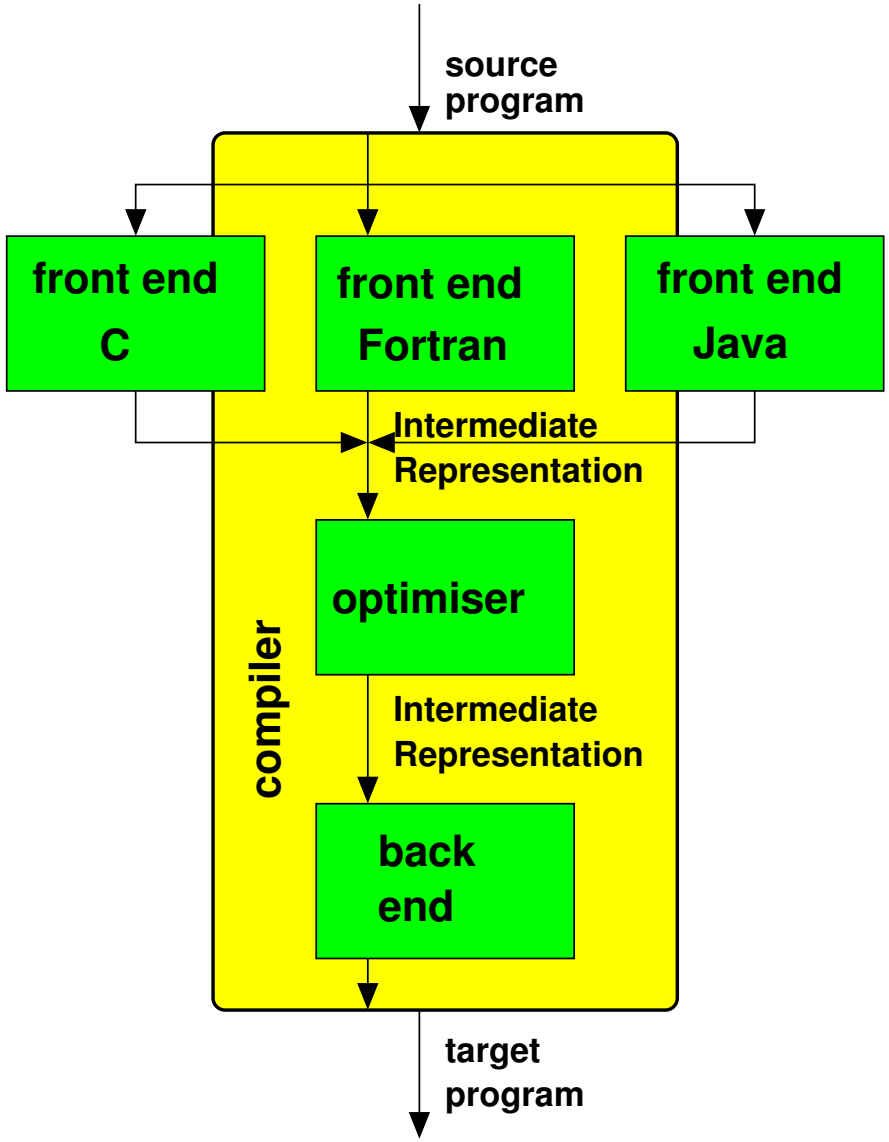
One Compiler — Many Backends



Rationale:

- Compiler backend is architecture specific
- Frontend and optimiser are generic
- Share large parts of compiler among different architectures

One Compiler — Many Frontends



Rationale:

- Compiler frontend language specific
- But concrete syntax removed during scanning/parsing
- Abstract syntax often similar across languages with similar feature set
- Reuse optimiser and backend for various languages

Desirable Properties of a Compiler

- **Run-time efficiency:**
  performance-critical applications,
  high-performance computing

Desirable Properties of a Compiler

- **Run-time efficiency:**
  performance-critical applications,
  high-performance computing
- **Space:**
  limited code/data memory, embedded systems,
  micro-controllers, digital signal processors

Desirable Properties of a Compiler

- **Run-time efficiency:**
  performance-critical applications,
  high-performance computing
- **Space:**
  limited code/data memory, embedded systems,
  micro-controllers, digital signal processors
- **Feedback:**
  verbose error messages, identification in source code,
  hints on correction

Desirable Properties of a Compiler

- **Run-time efficiency:**
  performance-critical applications,
  high-performance computing
- **Space:**
  limited code/data memory, embedded systems,
  micro-controllers, digital signal processors
- **Feedback:**
  verbose error messages, identification in source code,
  hints on correction
- **Debugging:**
  source-level debugging, execution as specified,
  no transformations

Desirable Properties of a Compiler

- **Run-time efficiency:**
  performance-critical applications,
  high-performance computing
- **Space:**
  limited code/data memory, embedded systems,
  micro-controllers, digital signal processors
- **Feedback:**
  verbose error messages, identification in source code,
  hints on correction
- **Debugging:**
  source-level debugging, execution as specified,
  no transformations
- **Compile-time efficiency:**
  short development cycles, agility in software engineering

The End: Questions ?

